# 2024 年第三届“钉钉杯”大学生
# 大数据挑战赛论文

题　　目　基于 LSTM 和 Neural Prophet 模型的门诊收入预测研究

## 摘　要

本文深入分析了智慧医疗对于提高医疗服务效率及促进医疗行业发展的重要贡献。在当前信息化技术迅速发展的背景下，智慧医疗已变成现代医疗资源分配、减少医疗开销、并增进病患满意度的关键因素。本研究以一家医院门诊的医疗数据为分析对象，探讨了门诊和住院收费系统在智慧医疗体系中所扮演的角色，并指出通过开发预测模型来优化医疗资源配置及提升服务效率是未来研究的重要方向。

**针对问题一：**我们采用了两种不同的时间序列预测模型：长短期记忆网络（LSTM）和 Neural Prophet 模型。在建模前，我们对相关数据，包括日期、患者数量、药品总收入和当日病房收入进行了彻底的预处理，处理内容包括填补缺失值以及识别并处理异常值，以确保数据的完整性和准确性。这两个模型均依据所提供的关键数据变量来进行拟合，最终，我们呈现了科室在空白数据集中门诊收入的预测结果。

**针对问题二：**与问题一相似，在针对康复医学科二病房的门诊收入预测中，我们同样应用了 LSTM 和 Neural Prophet 模型，并对相关数据执行了相同的预处理步骤，保证了数据输入的清洁度和一致性。这两个模型的建立和训练流程与康复医学科一病房相同，目标是利用一致的方法学预测第二病房的门诊收入数据，进一步验证模型的泛化能力和预测效果。正文中展示了科室空白数据集中的门诊收入预测结果。

**针对问题三：**为了进一步提高预测的精确度和鲁棒性，我们将 LSTM 和 Neural Prophet 两种模型结合起来，构建了一个集成学习模型。该模型综合了两种单一模型的预测结果，并使用康复医学科三病房的数据进行调整和训练。通过集成学习，我们希望能够充分利用不同模型的优势，提升门诊收入预测的准确性，并为医院的资源配置和决策提供更科学和精确的支持。

关键词：智慧医疗；收入预测；LSTM 模型；Neural Prophet 模型；集成学习

# 目录


一、 问题重述 ..... 1
1.1 问题背景 ..... 1
1.2 问题分析 ..... 1
二、模型假设与符号说明 ..... 3
2.1 模型假设 ..... 3
2.2 符号说明 ..... 4
三、数据处理 ..... 5
3.1 数据预处理 ..... 5
数据标准化处理 ..... 5
3.2 数据可视化 ..... 6
四、 模型的建立与求解 ..... 12
4.1 问题一模型的建立与求解 ..... 12
4.1.1 LSTM 模型描述 ..... 12
4.1.2 Neural Prophet 模型描述 ..... 18
4.2 问题二模型的建立与求解 ..... 24
4.2.1 LSTM 模型描述 ..... 25
4.2.2 Neural Prophet 模型描述 ..... 27
4.3 问题三模型的建立与求解 ..... 32
4.3.1 基于 Stacking 学习法构建集成学习模型描述 ..... 32
4.3.2 集成模型的建立与求解 ..... 33
五、 模型评价 ..... 37
5.1 模型优点和缺点 ..... 37
5.1.1 LSTM 模型的优点 ..... 37
5.1.2 LSTM 模型的缺点 ..... 37
5.1.3 Neural Prophet 模型的优点 ..... 38
5.1.4 Neural Prophet 模型的缺点 ..... 38
5.2 模型总结和展望 ..... 38
六、 参考文献 ..... 39
七、 附录 ..... 39

# 一、问题重述

## 1.1 问题背景

在当今社会，随着人口老龄化加剧和慢性病患者数量的上升，传统医疗系统面临着前所未有的挑战。这些挑战包括但不限于：管理系统的不完善、医疗成本的持续攀升、服务渠道的局限性以及医疗服务覆盖的不均衡性。这些问题不仅影响了患者的就医体验，也严重制约了医疗服务质量的提升，进而影响了公众的健康水平和社会的整体福祉。

智慧医疗的发展涉及多个方面，包括远程医疗、电子病历系统（EHRs）、智能诊断和预测模型等。通过智慧医疗，医院可以实现数据的实时监控和分析，从而更有效地管理病患和资源分配 (Wang et al., 2020)。为了有效应对这些挑战，智慧医疗的概念应运而生，并迅速成为全球医疗改革的重要方向。智慧医疗的核心在于利用现代信息技术，如大数据、云计算、人工智能、物联网等，构建一个高效、互联、共享的医疗信息网络平台体系。这个体系旨在通过数字化手段，实现医疗资源的优化配置，提供更加安全、便捷、优质的诊疗服务，同时显著降低患者的等待时间和医疗费用支出。

信息化的推进，使得医疗卫生服务如同得到了强大的助力，充分展现了信息技术在推动医疗改革和服务优化中的关键作用。从“互联网+医疗”到“AI+大健康”，再到“5G+智慧医疗”，技术的不断进步和应用模式的创新迭代，为医疗行业带来了前所未有的发展机遇。这些技术的应用不仅提高了医疗服务的效率和质量，更重要的是，它们极大地提升了公众的健康获得感、幸福感和安全感。

在智慧医疗的具体应用方面，门急诊收费系统和住院收费系统的建设尤为关键。门急诊收费系统通过高效的计算机应用程序来处理门急诊病人的划价、收费、退费、报销凭证打印等一系列流程，极大地提高了工作效率，改善了服务质量，并显著减少了患者的等待时间。而住院收费系统则专注于管理住院病人的费用信息，支持医院的经济核算和出院手续的办理，这不仅优化了医院的管理和服务流程，也在很大程度上减轻了医护人员的工作负担。

本文探讨了智慧医疗如何优化医疗资源和提升服务效率。通过分析医院门诊数据，研究强调门急诊和住院收费系统的重要性，并采用 LSTM 与 Neural Prophet 模型预测门诊收入，验证模型泛化能力。进一步，研究结合这两种模型构建集成学习模型，以提高预测精度，为改进医院资源配置提供支持。

## 1.2 问题分析

**问题一：**要求针对康复医学科第一病房，构建两个不同类型的时间序列预测模型来预测门诊收入。这个问题的关键在于充分利用给定的输入数据(日期、患者人数、药品总收入、当日病房收入)来构建有效的预测模型。

在解决康复医学科第一病房的门诊收入预测问题时，我们首先对所提供的数据进行

了彻底的清洗和特征工程处理。步骤包括处理缺失值和异常值，并对各指标进行统计分析，计算其平均值、最大值、最小值等关键统计量，以便识别和修正数据中的异常值及缺失值。

接下来，我们引入了时间相关的特征，如星期几和是否为节假日，因为这类因素对门诊量有显著影响。对于连续型变量，我们实施了归一化或标准化处理，确保模型可以公平地处理不同尺度的特征。在模型训练和评估阶段，我们合理划分了训练集和测试集，确保模型的泛化能力。

在评估模型性能时，我们采用了均方根误差（RMSE）、平均绝对误差（MAE）和平均绝对百分比误差（MAPE）等指标。同时，我们重视模型的可解释性，尤其是在医疗预测任务中非常关键。通过使用特征重要性分析和 SHAP 值，我们解释了模型的预测结果，帮助理解影响门诊收入的关键因素，并为医院管理决策提供了洞察。

此外，还关注模型的可解释性，特别是对于医疗相关的预测任务。我们还可以使用特征重要性分析或 SHAP(SHapley Additive exPlanations)值来解释模型的预测结果，这不仅有助于理解影响门诊收入的关键因素，也能为医院管理决策提供有价值的洞察。最后，对于最终选定的模型，建议进行敏感性分析，以了解模型对不同输入变量变化的响应程度，从而评估预测结果的稳定性和可靠性。

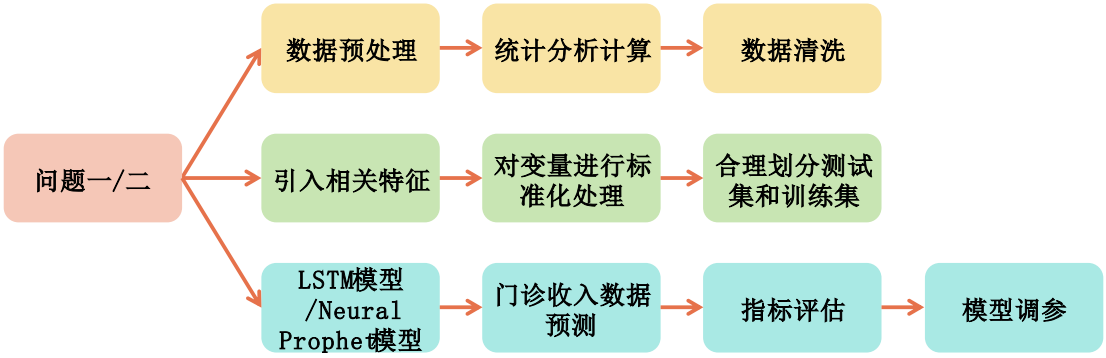



图 1 问题一和二的思路图

问题二：第二个问题要求针对康复医学科二病房，同样构建两个不同类型的时间序列预测模型来预测门诊收入。这个问题与第一个问题在本质上是相似的，但由于是针对不同的病房，存在一些特殊性需要考虑。在特征工程方面，除了第一个问题中提到的特征外，还可以考虑创建一些反映二病房特性的特征。考虑到不同病房之间存在相互影响，我们可以尝试引入其他病房的一些相关指标作为特征，如一病房的患者人数或门诊收入等。时间相关的特征也很重要，除了基本的时间特征外，我们还可以考虑创建一些周期性特征，如一年中的第几周、一个月中的第几天等，这些特征能够捕捉到更细粒度的时间模式。

在模型训练和优化阶段，我们可以采用一些高级的技术来提高模型性能。例如，可以使用集成学习方法，如 bagging 或 boosting，来组合多个基础模型，从而获得更稳定和准确的预测结果。对于深度学习模型，我们可以考虑使用注意力机制(AttentionMechanism)来帮助模型捕捉关键时间步的信息。

此外，我们还可以尝试使用自动机器学习(AutoML)工具，如 Auto-ARIMA 或 Auto-Sklearn，来自动选择最佳的模型结构和超参数。在评估模型性能时，除了常用的误差指标外，我们还可以考虑使用方向准确率(Directional Accuracy)来评估模型预测趋势的能力。最后，对于预测结果，建议进行置信区间分析，以提供更全面的预测信息，这对于医疗决策尤为重要，因为它能够帮助管理者了解预测的不确定性范围。

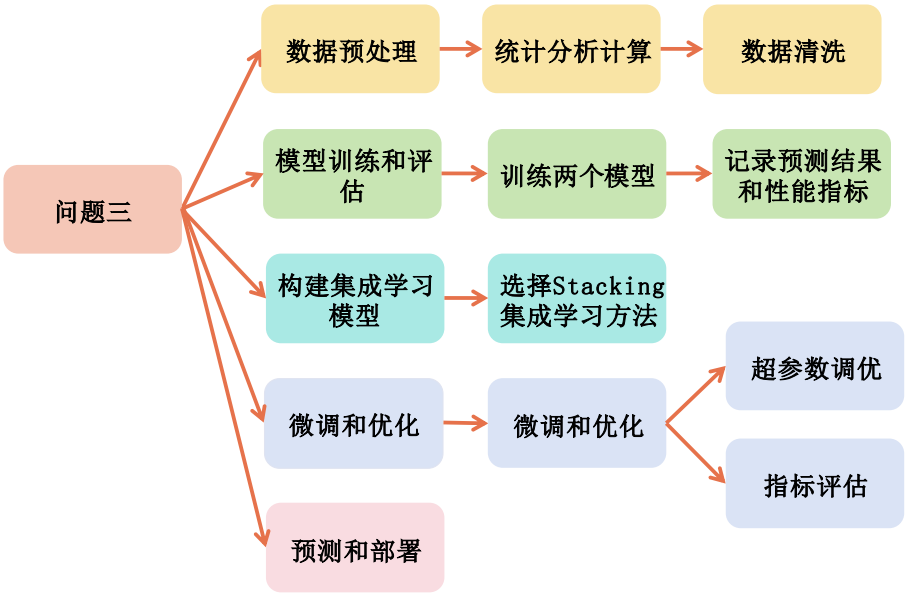



图 2 问题三的思路图

**问题三：**第三个问题要求结合前两个问题的模型，构建集成学习模型来预测康复医学科三病房的门诊收入。这是本题中最具挑战性的部分，需要综合运用前两个问题中的模型并引入新的集成学习技术。

集成学习的核心思想是通过组合多个基础模型来获得更好的预测性能。对于 stacking 模型，我们通常需要采用 k 折交叉验证来生成 meta 特征，以确保模型的稳定性。在评估集成模型的性能时，除了前面提到的各种指标外，我们还可以计算各个基础模型与集成模型之间的相关性，以分析集成学习的效果。

此外，考虑到医疗数据的特殊性，我们还应该关注模型的可解释性。可以使用 SHAP 值或部分依赖（Partial Dependence Plot)等技术来解释模型的预测结果，这不仅有助于理解影响门诊收入的关键因素，也能为医院管理决策提供有价值的洞察。最后，对于最终的预测结果，建议进行情景分析，模拟不同的外部环境变化(如政策调整、季节性流行病等)对门诊收入的影响。

# 二、模型假设与符号说明

## 2.1 模型假设

1）假设所选取的几百条数据具有随机性。

2）假设在进行问题建模分析的过程中不存在系统性风险。

3）假设提供的数据都是真实可靠的。

4）假设除了本文提到的影响因素外，没有其他因素影响模型的构建。

5）假设所提供的数据集在时间序列上是连续的，尽管存在一些空白数据点，但这些空白点不会对整体趋势产生显著影响。

6）假设经过适当的数据预处理后。门诊收入时间序列数据是平稳的，即其统计特性(如均值和方差）在时间上是恒定的。

7）假设康复医学科二病房的数据与一病房在结构和性质上是一致的，因此可以应用相同的预处理和建模方法。

8）假设数据中可能存在噪声。但这些操声不会对模型的预测能力产生根本性影响，通过活当的数据请洗和预处理可以减少嗓声的影响。

9）假设通过合理的模型训练和参数调整，所构建的模型能够达到较高的预测准确性，为医院的决策提供可靠的支持。

## 2.2 符号说明

表 1 符号说明

| 符号 | 说明 | 定义 |
|---|---|---|
| MSE | 均方误差 | 一种衡量预测模型准确性的指标，计算实际值与预测值之差的平方和的平均值。 |
| y | 优化目标函数 | 通常是机器学习或深度学习中需要最小化或最大化的函数。 |
| $x_i$ | 优化因素 | 指的是影响优化目标函数的变量或特征。 |
| $y_t$ | 时间序列数据 | 表示在特定时间点观察到的数据值。 |
| $f_t$ | 遗忘门 | 是循环神经网络（RNN）中长短时记忆网络（LSTM）的一个组成部分，用于控制信息的遗忘程度。 |
| $w_f$ | 遗忘门权重 | 是遗忘门中的一个参数，用于调整输入信息对遗忘门的影响程度。 |
| $b_f$ | 遗忘门偏置 | 是遗忘门中的一个参数，用于调整遗忘门的激活函数的偏移量。 |
| $h_{t-1}$ | 前一时间步隐藏状态 | 前一时间步隐藏状态，表示在当前时间步之前的隐藏状态，用于传递信息到当前时间步。 |
| $x_t$ | 时间步的输入 | 表示在当前时间步输入到模型的数据。 |
| $i_i$ | 输入门的输入 | 是循环神经网络（RNN）中长短时记忆网络（LSTM）的一个组成部分，用于控制新信息的输入程度。 |
| $w_i$ | 输入门的权重 | 是输入门中的一个参数，用于调整输入信息对输入门的影响程度。 |
| $b_i$ | 输入门的偏置 | 是输入门中的一个参数，用于调整输入门的激活函数的偏移量。 |
| $\tilde{C}_t$ | 记忆单元状态 | 是循环神经网络（RNN）中长短时记忆网络（LSTM）的一个组成部分，用于存储长期依赖信息。 |
| $O_t$ | 输出门的输出 | 是循环神经网络（RNN）中长短时记忆网络（LSTM）的一个组成部分，用于控制记忆单元状态对最终输出的影响程度。 |
| T(t) | 趋势部分 | 表示时间序列数据中的长期趋势成分。 |
| S(t) | 季节部分 | 表示时间序列数据中的季节性变化成分。 |
| E(t) | 自回归项 | 表示时间序列数据中的自回归成分，即当前值与过去值之间的关系。 |
| F(t) | 特殊事件 | 表示时间序列数据中受到特殊事件影响的成分。 |

| | | |
|---|---|---|
| A(t) | 滞后回归项 | 表示时间序列数据中的滞后回归成分，即当前值与过去一段时间内值的平均关系。 |
| L(t) | 未来回归项 | 表示时间序列数据中的未来回归成分，即当前值与未来一段时间内值的平均关系。 |

# 三、数据处理

## 3.1 数据预处理

在数据分析的流程中，数据预处理阶段对于建立准确且高效的预测模型是至关重要的环节。预处理的主要目标是将原始数据转化为更易于模型识别和处理的形式，以此优化模型的预测性能。在预处理之前，通过进行探索性数据分析来深入理解数据，包括数据分布、缺失值和异常值等，这些探索性分析的结果有助于选择最恰当的预处理策略。

接下来进行的是数据清洗。在这个阶段，移除了使用时长、上行流量和下行流量为 0 的数据，以免这些潜在的异常值对后续分析造成影响。同时，也剔除了使用时长过短（小于 5 秒）的数据，这部分数据可能由于用户误操作或者非常规使用行为产生，不适宜纳入分析范围。

## 数据标准化处理

为提高数据处理和分析的效率并有效消除量纲的影响，采用最大最小值标准化，将所有数据标准化到[0,1]区间中。以 x 表示原始数据，对数据标准化处理，确定各指标的最大值 Max 和最小值 Min，标准化公式如下

数据量化处理 APP 所属类别 a,b,c,数据为定类数据，不易对其进行描述，进行量化处理，将定类数据转换为定量数据。将类别标签转换为数值标签，即将 a,b,c,替换为 1 至 20 的数字。

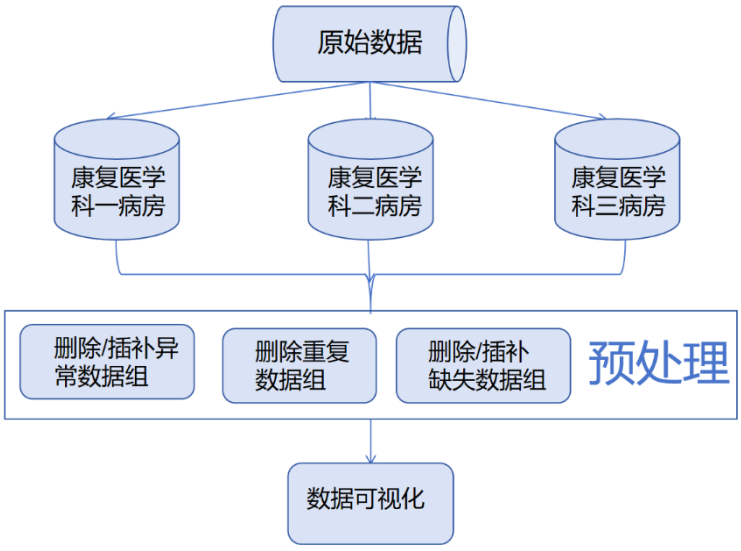



图 3 数据预处理过程

## 3.2 数据可视化

（一）描述性统计

在进行数据分析之前，本文首先对数据进行描述性统计。表 xx、表 xx 和表 xx 分别展示了康复医学科一病房、二病房和三病房的描述性统计结果。

表 2 显示，康复医学科一病房的患者人数分布较为广泛，较高的标准差表明患者人数波动显著。门诊收入平均值为 404962.2，显示出相对较高的收入水平，但门诊收入、药品总收入和当日病房收入的标准差较大，说明收入波动较大。

表 3 显示，康复医学科二病房的患者人数波动和三类收入的波动均较小，其中当日病房收入标准差最小，显示出较为稳定的收入情况。

表 4 显示，康复医学科三病房的患者人数波动和门诊收入波动幅度较大，但药品总收入和当日病房收入波动与二病房相近，显示出相对较低的波动性。

综合分析，各病房的门诊收入和患者人数均显示出不同的波动性和趋势。一病房和三病房的收入波动较大，可能与患者的就医需求波动有关；而二病房的收入则相对稳定，显示出较强的吸引力。整体来看，三病房的患者人数和收入波动幅度最大。

表 2 “康复医学科一病房”描述性统计结果

| 变量名 | 样本量 | 最大值 | 最小值 | 平均值 | 标准差 | 中位数 | 方差 | 峰度 | 偏度 | 变异系数（CV） |
|---|---|---|---|---|---|---|---|---|---|---|
| 门诊患者人次数 | 130 | 2562 | 446 | 1355.046 | 448.505 | 1285 | 201156.463 | 0.031 | 0.469 | 0.331 |
| 门诊收入 OBS_T01_MZSR68 | 115 | 1130102.42 | 58660.63 | 404962.213 | 176560.613 | 393477.6 | 31173650040.359 | 2.107 | 0.946 | 0.436 |
| 药品总收入 | 130 | 937271.26 | 40305.22 | 234669.475 | 146173.493 | 219282.815 | 21366690081.58 | 7.403 | 2.313 | 0.623 |
| 当日病房收入 | 130 | 69646.18 | -223.17 | 26426.784 | 17565.403 | 28238.36 | 308543378.79 | -0.884 | 0.324 | 0.665 |

表 3 “康复医学科二病房”描述性统计结果

| 变量名 | 样本量 | 最大值 | 最小值 | 平均值 | 标准差 | 中位数 | 方差 | 峰度 | 偏度 | 变异系数（CV） |
|---|---|---|---|---|---|---|---|---|---|---|
| 门诊患者人次数 | 127 | 2935 | 310 | 1397.213 | 452.21 | 1357 | 204493.724 | 1.228 | 0.366 | 0.324 |
| 门诊收入 OBS_T01_MZSR68 | 112 | 1431430.56 | 32064.22 | 390551.57 | 183536.243 | 364019.375 | 33685552424.631 | 9.492 | 2.059 | 0.47 |
| 药品总收入 | 127 | 1122088.56 | 19123.96 | 220595.819 | 132057.121 | 211593.13 | 17439083125.756 | 18.978 | 3.323 | 0.599 |
| 当日病房收入 | 127 | 51534.44 | 86.9 | 22721.226 | 14226.281 | 21130.72 | 202387079.225 | -1.169 | 0.141 | 0.626 |

表 4 “康复医学科三病房”描述性统计结果

| 变量名 | 样本量 | 最大值 | 最小值 | 平均值 | 标准差 | 中位数 | 方差 | 峰度 | 偏度 | 变异系数（CV） |
|---|---|---|---|---|---|---|---|---|---|---|
| 门诊患者人次数 | 101 | 2300 | 395 | 1362.149 | 428.344 | 1313 | 183478.328 | -0.436 | 0.093 | 0.314 |

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 门诊收入 OBS_T01_MZSR68 | 86 | 705407.05 | 47988.79 | 349046.741 | 127053.757 | 330723.245 | 16142657268.593 | -0.143 | 0.291 | 0.364 |
| 药品总收入 | 101 | 525087.9 | 49961.04 | 193510.449 | 90238.835 | 190889.18 | 8143047305.815 | 1.868 | 1.024 | 0.466 |
| 当日病房收入 | 101 | 55297.39 | 388.8 | 21464.35 | 15115.831 | 17338.13 | 228488353.839 | -1.163 | 0.436 | 0.704 |

（二）散点图

图 4、图 5 和图 66 分别为康复医学科一病房、二病房和三病房的散点图。

整体来看，康复医学科一病房的患者人数和收入相对稳定，但存在较大波动；二病房表现出良好的稳定性，患者量与收入呈正相关关系；三病房的收入和患者人数均有波动。

图 4 显示，康复医学科一病房的门诊患者人次数散点分布较广，患者人数在 600 到 2400 之间。大部分数据点集中在 1000 到 1500 之间，显示出患者数量的相对稳定性，但也有一些高峰。门诊收入范围从 100,000 到 800,000 不等，整体趋势显示随着患者数量的增加，收入也有所上升。一些高收入的异常值表明特定病例可能带来了较高的收入。药品总收入散点分布相对集中，药品收入在 100,000 到 600,000 之间。大部分数据点集中在 200,000 到 400,000，显示出药品收入的稳定性。当日病房收入范围较广，部分数据点显示出较高的收入（如接近 70,000），整体趋势较为平稳，但有一些波动，需关注。

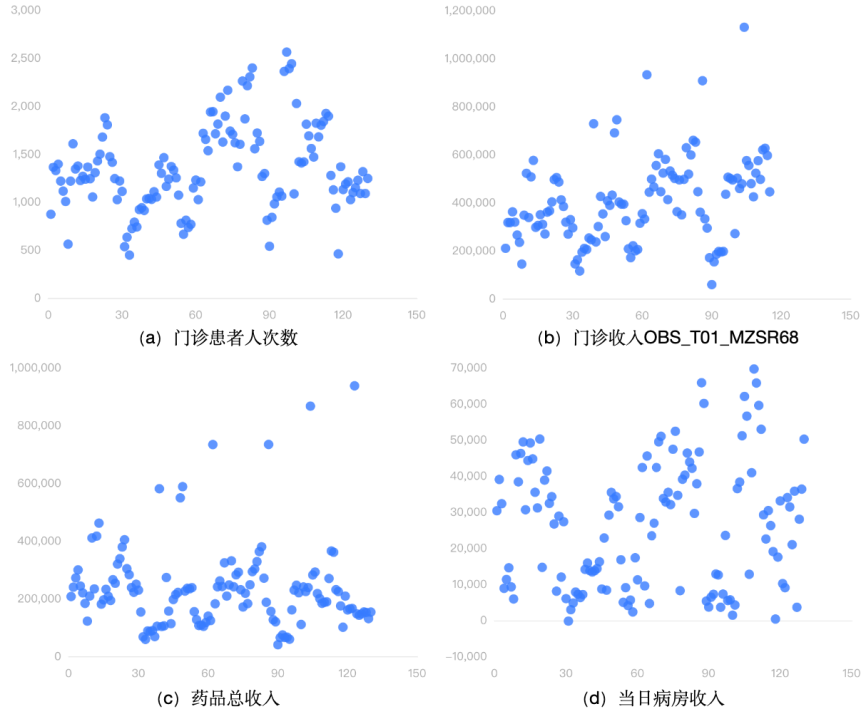



图 4 康复医学科一病房散点图

图 5 显示，康复医学科二病房的门诊患者人次数散点分布较为集中，患者人数在 800 到 2100 之间。数据点较为均匀，显示出该病房的患者量相对稳定。门诊收入主要集中在 200,000 到 600,000 之间，整体趋势向上，显示出随着患者人数的增加，收入也逐渐上升。药品总收入范围较广，集中在 100,000 到 600,000 之间。大部分数据点显示出药品收入的稳定性和一定的波动性。当日病房收入较为集中，显示出一定的波动性，主要

集中在 20,000 到 60,000 之间，整体上表现出较为稳定的收入情况。

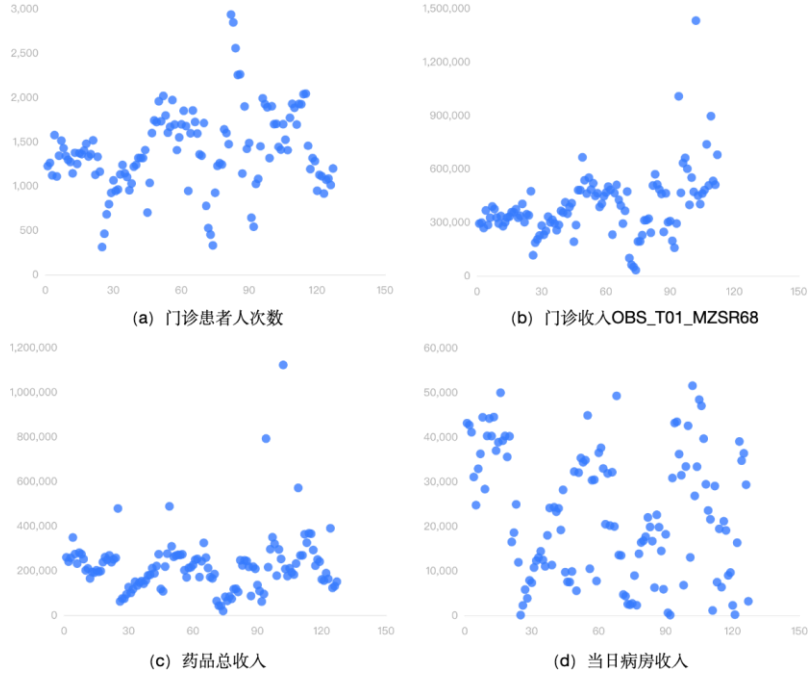



图 5 康复医学科二病房散点图

图 6 显示，康复医学科三病房的门诊患者人次数散点分布较为宽泛，患者人数在 600 到 2400 之间。大部分数据点集中在 1000 到 1600 之间，显示出一定的稳定性。门诊收入范围从 100,000 到 900,000 不等，整体趋势显示出一定的上升性。一些高收入的异常值说明特定病例可能带来了较高的收入。药品总收入分布较广，主要集中在 100,000 到 600,000 之间，显示出药品使用的波动性，部分数据点显示出较高的药品收入。当日病房收入波动较大，范围大致在 0 到 70,000 之间，显示出该病房当日收入的波动性。

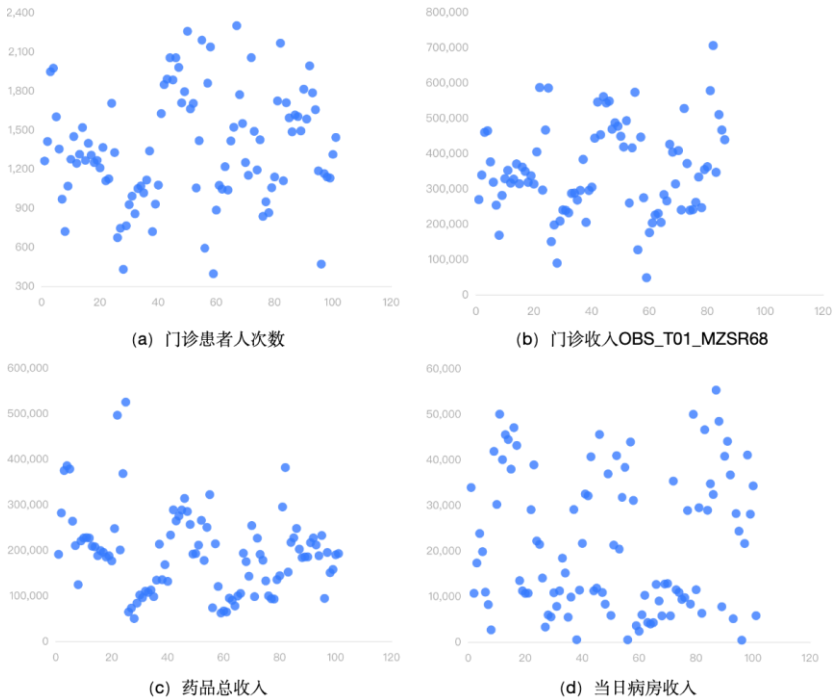



图 6 康复医学科三病房散点图

（三）箱型图

图 7、图 8 和图 9 分别为康复医学科一病房、二病房和三病房的箱型图。

整体来看，康复医学科一病房的患者人次数和收入波动较大；二病房表现出良好的稳定性，患者量和收入相对集中；三病房的收入潜力大。

图 7 显示，康复医学科一病房的门诊患者人次数的中位数约为 1200，四分位数范围在 600 到 1500 之间，显示出患者人数的相对集中。存在一些异常值，可能是由于个别病例或特殊情况导致的高患者人数。门诊收入的中位数接近 400,000，四分位数范围在 200,000 到 800,000 之间。收入分布较广，上方有几个异常值，表明存在一些高收入的病例。药品总收入的中位数约为 200,000，四分位数范围在 100,000 到 400,000 之间。整体收入分布相对稳定，但也有一些高收入的异常值。当日病房收入的中位数接近 30,000，四分位数范围在 10,000 到 60,000 之间。收入波动性较小，显示出该病房的日常收入相对稳定。

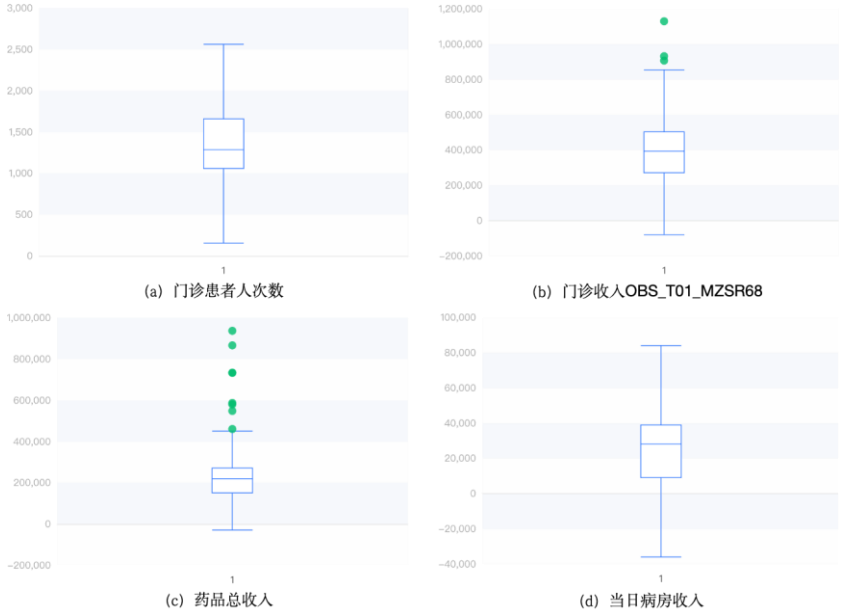



图 7 康复医学科一病房箱型图

图 8 显示，康复医学科二病房的门诊患者人次数的中位数约为 1500，四分位数范围在 900 到 2100 之间，患者人数相对集中。同样存在一些异常值，可能与特定患者的治疗需求相关。门诊收入的中位数在 400,000 左右，四分位数范围在 200,000 到 600,000 之间。收入分布较为集中，显示出良好的经济效益，异常值数量较少。药品总收入的中位数约为 400,000，四分位数范围在 200,000 到 600,000 之间。药品收入的波动性较小，整体表现出稳定的收入水平。当日病房收入的中位数约为 20,000，四分位数范围在 10,000 到 40,000 之间。收入波动性相对较小，显示出该病房的日常运营较为平稳。

图 9 显示，康复医学科三病房的门诊患者人次数的中位数约为 1500，四分位数范围在 800 到 2000 之间，患者人数相对集中。包含几个异常值，可能与特定病例相关。门诊收入的中位数接近 500,000，四分位数范围在 300,000 到 900,000 之间。收入分布较广，存在一些高收入的异常值，显示出该病房的收入潜力。药品总收入的中位数接近 400,000，

四分位数范围在 200,000 到 600,000 之间。显示出药品收入的稳定性，但也有一些高收入的异常值。当日病房收入的中位数约为 20,000，四分位数范围在 0 到 60,000 之间。收入波动性相对较小，整体表现出稳定的日常收入。

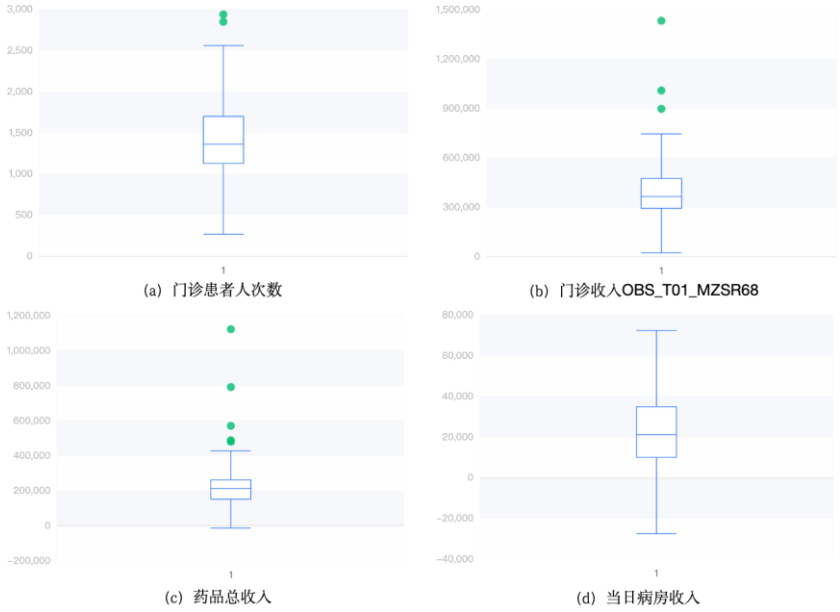



图 8 康复医学科二病房箱型图

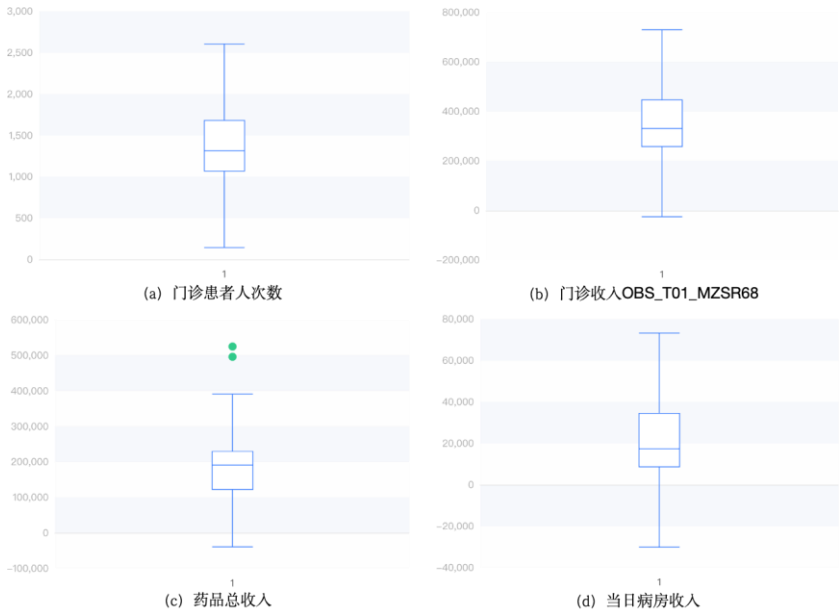



图 9 康复医学科三病房箱型图

（四）趋势分析

图 10、图 11 和图 12 分别为康复医学科一病房、二病房和三病房的折线图。

根据康复医学科一病房、二病房和三病房的门诊收入折线图，通过分析各病房门诊收入随日期的变化趋势，本文发现门诊收入与季节存在一定关联性。

整体来看，各病房门诊收入与季节的关联性存在以下特征：

1.季节性高峰

（1）冬季高峰：三病房和一病房在冬季（如 12 月到次年 2 月）呈现出明显的门诊收入高峰。这可能与冬季常见的流感、感冒等疾病的增加有关。

（2）春季与秋季波动：春季和秋季（如 3 月和 9 月）也出现了一定的门诊收入波动，可能与换季时人们健康问题的增加有关。

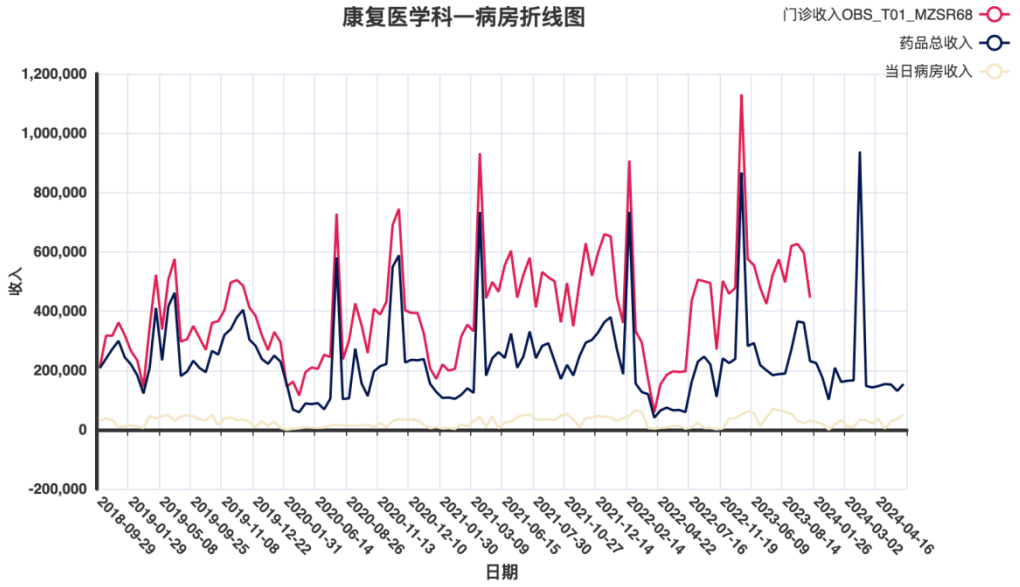



图 10 “康复医学科一病房”折线图

2.夏季低谷

所有病房在夏季（如 6 月到 8 月）时，门诊收入普遍较低。这可能与人们的外出旅游和就医需求减少有关。

3.年度变化趋势

各病房的门诊收入呈现出相似的年度波动模式，尤其是在年末和年初，收入通常会出现高峰。三个病房在不同年份的收入变化趋势也较为一致，反映出季节性因素对门诊收入的影响。

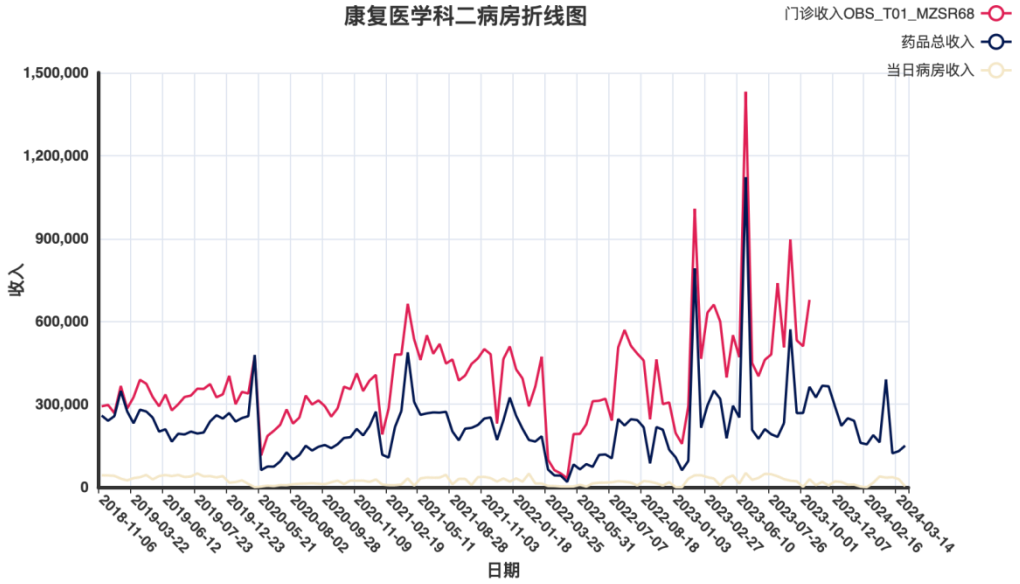


图 11 “康复医学科二病房”折线图

4.疫情影响

从 2020 年开始，疫情对各病房的门诊收入产生了显著影响，导致收入波动加大，尤其是在疫情初期。随着疫情的控制，各病房的门诊收入逐渐恢复，但仍受季节性因素的影响。

5.长期趋势

从整体来看，三病房的门诊收入在 2018 年至 2024 年间都呈现出逐渐上升的趋势，表明康复医学科的需求在持续增长。

康复医学科各病房的门诊收入与季节之间存在明显的关联性，受季节性疾病流行、社会经济变化及突发公共卫生事件等多种因素的影响。冬季的高峰和夏季的低谷是主要的季节性特征。

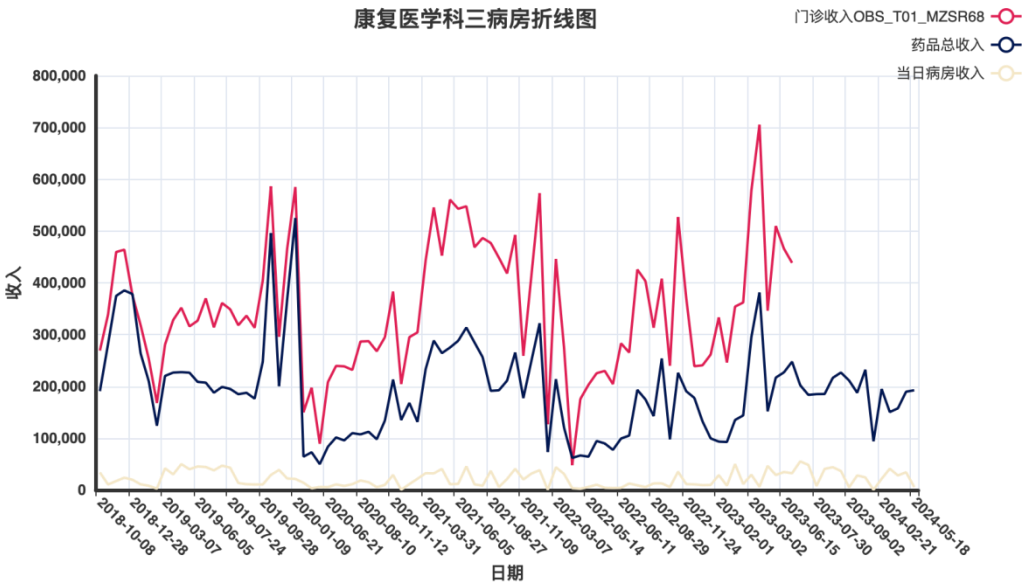



图 12 “康复医学科三病房”折线图

# 四、模型的建立与求解

## 4.1 问题一模型的建立与求解

### 4.1.1 LSTM 模型描述

在本次实验中，我们使用了 LSTM（长短期记忆）神经网络模型来预测康复医学科一病房的门诊收入。长短期记忆网络（Long Short-Term Memory，简称 LSTM）是一种针对解决循环神经网络（RNN）中梯度消失和梯度爆炸问题的特殊结构。LSTM 网络通过门控能对信息的保留和遗忘，使其在处理长期依赖关系时具有较强的记忆能力，在医疗收入预测中的应用日益增多。例如，研究表明，LSTM 在预测每日住院人数和收入方面表现出较高的准确性 (Shi et al., 2018)。

LSTM 是一种特殊的递归神经网络（RNN），能够捕捉和保留长时间依赖关系，特别适用于处理和预测时间序列数据。在医疗收入预测中，LSTM 模型被广泛应用于各种数据的预测，如患者数量、药品收入和住院费用等。

LSTM 在医疗预测中有着广泛的应用：

患者流量预测：研究表明，LSTM 模型能够准确预测医院的每日住院和门诊流量，从而帮助医院更好地管理资源配置 (Choi et al., 2018)。

医疗费用预测：LSTM 模型被用于预测住院费用和门诊收入，其高准确性和鲁棒性使其成为医疗收入预测的重要工具 (Xiao et al., 2019)。

疾病进展预测：LSTM 在预测慢性疾病（如糖尿病和心脏病）患者的病情进展方面也表现出色，有助于制定个性化的治疗方案 (Lipton et al., 2015)。

在我们的问题中，需要将门诊患者人次数、药品总收入和当日病房收入联合起来预测与门诊收入的相关性，应用 LSTM 神经网络，使得模型在处理长期依赖关系时具有较强的记忆能力具有较好的拟合效果。通过门控机制的设计，允许信息根据上下文的重要程度进行选择性的存取和遗忘，有效地解决了长期依赖问题。这对于理解医疗趋势，如慢性病的发展趋势非常有用。

通过训练数据，LSTM 可以学习并捕捉案例医疗数据中的特定模式和关系，并根据任务完成情况进行权重的优化调整。可以充分兼顾到医疗数据中的多维指标。

LSTM 模型网络结构图如图 4 所示。

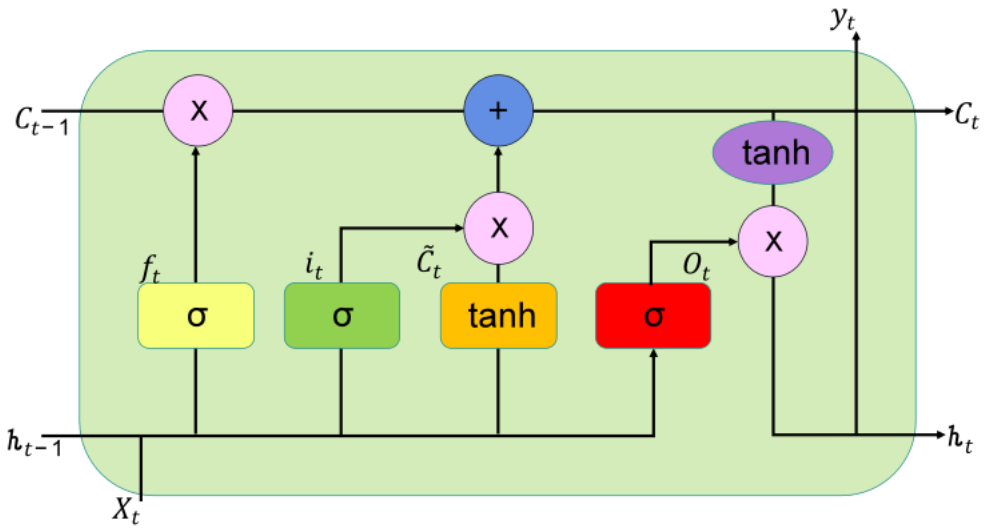



图 13 LSTM 神经网络原理图

LSTM 的核心是由三个门控单元组成，分别是遗忘门（Forget Gate）、输入门（Input Gate）和输出门（Output Gate）。门的设计使得 LSTM 可以有选择地保存和更新信息，从面有效地处理长时间间隔的依赖关系。

LSTM 单元在时间 t 的状态计算如下，遗忘门：

$$f_t = \sigma\left(W_f \times [h_{t-1}, x_t] + b_f\right) \tag{1}$$

其中$f_t$是遗忘门的输出，$\sigma$是 sigmoid 函数，$w_f$和$b_f$是遗忘门的权重和偏置，$h_{t-1}$是

前一时间步的隐藏状态，$x_t$是当前时间步的输入。遗忘门通过一个 sigmoid 激活函数（$\sigma$）决定是否丢弃记忆单元中的信息。

输入门：

$$i_t = \sigma(W_i \times [h_{t-1}, x_t] + b_i) \tag{2}$$

候选记忆单元：

$$\tilde{\tilde{C}}_t = tanh(W_c \times [h_{t-1}, x_t] + b_c) \tag{3}$$

记忆单元：

$$\tilde{C}_t = f_t \times \tilde{C}_{t-1} + i_t \times \tilde{\tilde{C}}_t \tag{4}$$

输入门通过一个 sigmoid 激活函数决定要更新记忆单元的哪些部分和输入的哪些部分。输入为上一个时间步的隐藏状态（$h_{t-1}$）和当前时间步的输入（$x_t$），经过线性变换和 sigmoid 激活函数计算得到输入门的输出（$i_t$）。 输入门的输出还通过 tanh 激活函数计算得到新的候选细胞记忆（$\tilde{C}_t$）。输入门的输出和新的候选细胞记忆相乘，并与遗忘门的输出相加，用于更新当前时间步的细胞状态。

输出门:

$$o_t = \sigma(W_o.[h_{t-1}, x_t] + b_o) \tag{5}$$

隐藏状态:

$$h_t = o_t \times tanh(C_t) \tag{6}$$

输出门（Output Gate）：输出门通过一个 sigmoid 激活函数决定细胞状态中哪些部分将被输出。输入为上一个时间步的隐藏状态（$h_{t-1}$）和当前时间步的输入（$x_t$），经过线性变换和 sigmoid 激活函数计算得到输出门的输出（$O_t$）。输出门的输出经过 tanh 激活函数计算得到当前时间步的隐藏状态（$h_t$）。细胞状态经过 tanh 激活函数计算得到当前时间步的输出（$y_t$）,与输出门的输出相乘得到最终输出。

综上所述，其流程图如下：

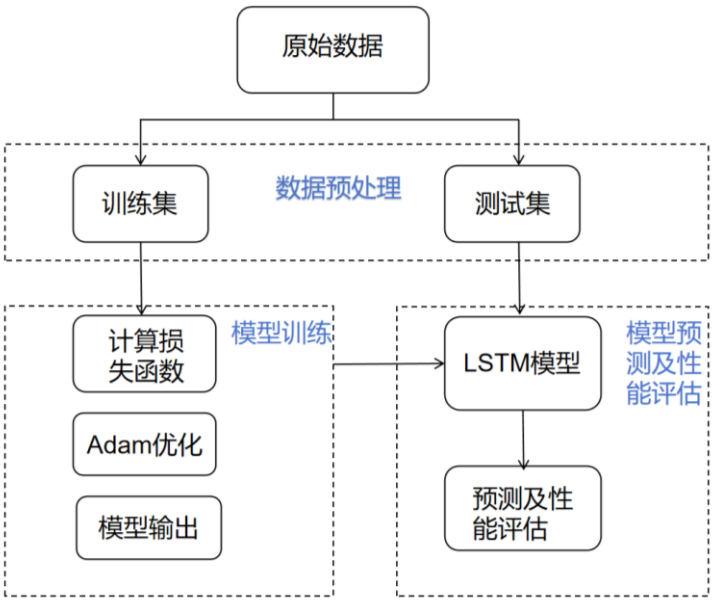



图 14 门诊医疗收入的 LSTM 模型流程图

**1. 数据加载和预处理**

我们首先加载并预处理了数据。数据来自某医院的门诊医疗记录，包含门诊患者人次数、药品总收入、当日病房收入等变量。为了进行分析和预测，我们选择了康复医学科一病房的数据。我们对数据进行了初步的筛选和清洗，去除了负值和异常值，确保每一行数据的门诊患者人次数、药品总收入和当日病房收入均大于等于零。同时，我们将日期字段转换为 datetime 类型，并设置为数据的索引。

**2. 数据插值和标准化**

为了解决数据中的缺失值问题，我们使用时间插值法填充了缺失的“门诊收入 OBS_T01_MZSR68”字段数据。接着，为了适应 LSTM 模型的输入需求，我们对数据进行了标准化处理。标准化过程使用了 MinMaxScaler，将数据缩放到 0 和 1 之间。这样做的目的是为了加快模型训练速度并提高模型预测性能。

**3. 数据转换为 LSTM 输入格式**

我们将标准化后的数据转换为适合 LSTM 模型输入的形状。具体来说，我们定义了一个滑动窗口，将时间序列数据切分为特定长度的子序列。每个子序列作为 LSTM 模型的一个输入样本。我们将整个数据集划分为训练集和测试集，其中 70%的数据用于训练，30%的数据用于测试。

**4. 构建和训练 LSTM 模型**

为了构建预测模型，我们使用了 Long Short-Term Memory (LSTM) 神经网络。LSTM 是一种特殊的递归神经网络（RNN），擅长处理和预测时间序列数据。我们的 LSTM 模型由多层 LSTM 单元组成，以提高模型的表达能力和预测精度。具体来说，我们的模型结构如下：

**第一层：**一个具有 256 个隐藏单元的 LSTM 层。这个层用于捕捉输入时间序列中的长程依赖关系。

**第二层：**一个 Dropout 层，丢弃比例为 0.2，用于防止过拟合。Dropout 层随机地将输入单元设置为零，以减少过拟合现象。

**第三层：**一个具有 128 个隐藏单元的 LSTM 层，进一步提取时间序列中的特征。

**第四层：**另一个 Dropout 层，丢弃比例为 0.2，再次用于防止过拟合。

**第五层：**一个具有 64 个隐藏单元的 LSTM 层，继续提取特征。

**第六层：**最后一个 Dropout 层，丢弃比例为 0.2。

**输出层：**一个全连接层，输出一个用于预测门诊收入的标量值。

模型使用 adam 优化器进行优化，并使用均方误差（MSE）作为损失函数进行编译。adam 优化器是一种基于一阶和二阶动量的优化算法，具有较快的收敛速度和较好的效果。均方误差是常用的回归模型损失函数，用于衡量预测值与实际值之间的差距。

训练过程中，我们将模型在训练数据上进行了 150 个 epoch 的训练。每个 epoch 都包含了一个完整的前向传递和反向传播过程，通过不断调整模型的权重来最小化损失函数。为了进一步防止过拟合，我们在训练过程中使用了 EarlyStopping 回调函数，该函数

会在模型性能不再提升时提前停止训练。

5. 模型预测

训练完成后，我们使用模型对训练集和测试集进行了预测。由于 LSTM 模型的输出是标准化的值，因此为了便于结果解释，我们对预测值进行了反向转换，将其恢复到原始数据的尺度。这样做可以使预测结果更具实际意义。

**6. 分类指标计算**

为了进一步评估模型的性能，我们计算了测试数据上的分类指标。

我们计算了训练集和测试集的均方误差（MSE），分别为 0.02 和 0.03。MSE 是衡量回归模型性能的一个常用指标，计算的是预测值与实际值之差的平方的平均值。均方误差 MSE 越小，表明模型的预测值越接近实际值，模型的预测性能越好。训练集和测试集的 MSE 较低，表明模型能够较好地拟合训练数据，并在测试数据上进行合理的预测。

通过设定动态阈值，我们将预测值与实际值之间的误差转换为二元分类问题。具体来说，如果预测值与实际值之间的误差在某一阈值范围内，则认为预测是准确的，否则认为预测是错误的。

在此基础上，我们计算了以下分类指标：

- **准确率（Accuracy）**：这是一个基本的性能度量，表示模型正确预测的样本数占总样本数的比例。准确率越高，模型的性能通常被认为越好。然而，准确率可能会受到数据集中类别不平衡的影响，因此在类别分布不均的情况下可能不是最佳的评价指标。

  模型在测试数据上的准确率为 0.9286，表明模型能够正确预测大部分测试样本。
- **F1-score**：F1 分数是准确率和召回率的调和平均数，它试图平衡这两个指标。准确率指的是预测为正的样本中真正为正的比例，而召回率则是所有实际为正的样本中被正确预测为正的比例。F1 分数综合考虑了模型的精确性和覆盖度，是一个更为全面的评估指标，尤其是在类别不平衡的数据集中。

  模型在测试数据上的 F1-score 为 0.963，F1-score 是准确率和召回率的调和平均值，能够更全面地反映模型的性能。

根据上面指标，可以看出模型在训练集和测试集上的预测效果较为理想，说明模型能够较好地拟合训练数据并对测试数据进行合理的预测。

7. 预测结果可视化

为了更直观地展示模型的性能，我们绘制了预测结果图。图中展示了实际值、训练预测值和测试预测值。通过图像可以看出，模型能够较好地拟合实际数据，并在测试数据上具有良好的预测性能。

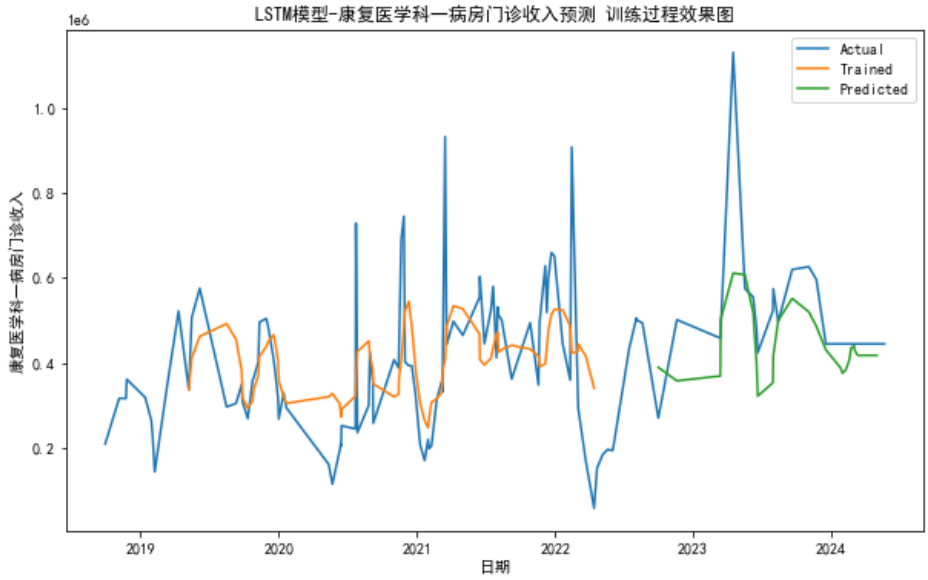



图 15 康复医学科一病房的 LSTM 模型训练过程

同时，我们还绘制了模型训练过程中的损失函数变化图，展示了训练集和验证集的损失函数随着训练迭代次数的变化情况。可以看出，随着训练的进行，损失函数逐渐减小，并趋于稳定，表明模型收敛效果良好。

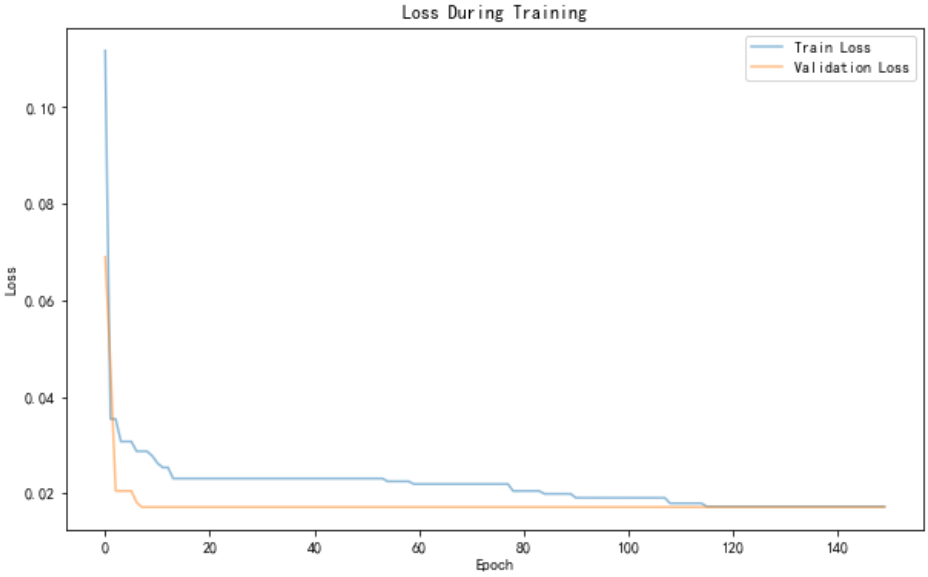



图 16 康复医学科一病房的 LSTM 模型损失函数

8. 保存预测结果

最后，我们将预测结果保存到 Excel 文件中，以便后续分析和使用。预测结果文件包含了日期、实际值、预测值以及各类回归变量等信息。通过这些预测结果，我们可以更好地了解未来门诊收入的变化趋势，并为相关决策提供数据支持。

表 5 康复医学科一病房的 LSTM 预测收入结果

| 日期 | 预测收入 | 日期 | 预测收入 |
|---|---|---|---|
| 2024-01-26 | 520255.78 | 2024-03-13 | 440229.50 |
| 2024-01-31 | 486188.87 | 2024-03-19 | 430457.18 |
| 2024-02-09 | 431067.03 | 2024-04-16 | 418070.15 |
| 2024-02-18 | 388698.0 | 2024-05-01 | 418070.15 |
| 2024-02-23 | 376125.06 | 2024-05-06 | 418070.15 |
| 2024-03-02 | 383884.46 | 2024-05-09 | 418070.15 |
| 2024-03-03 | 406730.15 | 2024-05-21 | 418070.15 |
| 2024-03-11 | 435676.06 | | |

### 4.1.2 Neural Prophet 模型描述

Neural Prophet 模型是基于经典 Prophet 的改进版本，采用了神经网络结构，使用 PyTorch 实现。Neural Prophet 模型能够处理复杂的时间序列数据，并在医疗收入预测中展现出优异的性能。例如，利用 Neural Prophet 模型进行的医院收入预测研究显示，其预测精度显著高于传统的时间序列预测方法 (Taylor & Letham, 2018)。

Neural Prophet 在医疗收入预测中的应用：

季节性数据预测：Neural Prophet 模型能够有效处理医疗收入中的季节性和周期性变化，其预测结果在多个研究中表现出色 (Taylor & Letham, 2018)。

多变量时间序列预测：该模型可以处理多个输入变量，提高预测的精确性和适用性。例如，利用 Neural Prophet 进行的医院多变量收入预测显示出高精度和稳定性 (Sygel et al., 2021)。

数据自动化处理：Neural Prophet 的自动化建模过程使其在数据预处理和模型选择方面具有显著优势，适用于大规模医疗数据分析 (Blumenstock et al., 2021)。

Prophet 专注于分析时序数据（医疗数据）的特征和变化规律，并用于预测未来走势。相较于传统时序模型，Prophet 不仅能够有效处理数据缺失和模型灵活性问题，而且在预测准确率和用户交互方面表现更为优异。Prophet 算法是一个可分解的时间序列模型，它对时序数据各个特征进行分析，通过将几项特征拟合，累加起来得到时间序列的预测值。一个时间序列往往是几类变化形式的叠加或耦合：长期趋势(Secular trend)，季节变动(Seasonal Variation)，不规则波动(Irregular Variation)。对于我们研究的医疗数据特征而言比较符合。也就是说对所有的 t>=0，都有：

$$y_t = T_t + S_t + I_t \tag{7}$$

除了加法形式还有乘法形式，Prophet 就是在加法模型的基础上进行了必要的改进和优化。也就是：

$$y_t = T_t \times S_t \times I_t \tag{8}$$

因为在我们本文中，医疗机构的运营数据除了有趋势项、季节项、剩余项之外，通常还有节假日的效应。所以，在 prophet 算法里面，考虑了以上 4 项，即：

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \tag{9}$$

其中，g(t)为趋势项，它表示时间序列在非周期上面的变化趋势，即排除其他因素后呈现出的趋势；s(t)为周期项，或者成为季节项，一般病患看某种病症可能会有季节性的趋势；h(t)为节假日项，表示在当天是否存在节假日，这也是病患看病与否的一种因素；最后一项为误差项、噪声项或称为剩余项，表示模型未预测到的波动。Prophet 算法通过拟合这几项，然后把它们累乘起来得到时间序列预测值。

（1）趋势项 g(t)：

g(t)在 prophet 算法里，增长趋势是整个模型的核心组件，用于描述整个时间序列的增长趋势和未来的预测走势。诸如医疗机构的运营数据往往会呈现出明显的趋势性。这部分提供了两种趋势函数：Non-linear growth（非线性增长）和 Linear growth（线性增长）。Non-linear growth 非线性增长是基于逻辑回归的趋势项。如果认为时间序列的整体增长趋势是线性的（Linear growth），那么就可以采用线性模型：

$$g(t) = (k + a(t)^T\delta)t + (m + a(t)^T\gamma) \tag{70}$$

$$\gamma = (\gamma_1, \dots\dots, \gamma_S)^T, \gamma_j = -s_j\delta_j \tag{11}$$

（2）周期项 s(t)：

几乎所有的时间序列预测模型都会考虑这个因素，因为时间序列通常会随着天、周、月、年等季节性的变化而呈现季节性变化，即周期性的变化。医疗机构的运营数据往往会呈现出的趋势和季节性。在数学上，周期性函数可以用正弦和余弦函数的傅立叶级数表示：

$$s(t) = \sum_{n=1}^{N}\left(a_n cos\left(\frac{2\pi nt}{P}\right) + b_n sin\left(\frac{2\pi nt}{P}\right)\right) \tag{18}$$

$$\beta = (a_1, b_1, \dots\dots, \quad a_N, b_N)^T \tag{19}$$

Neural Prophet 模型的预测流程如下图所示：

（3）节假日项 h(t) ：

节假日或重大事件通常会对数据产生显著影响，而且这些时间点往往不具有周期性。因此，对这些特殊事件进行分析是非常必要的，有时甚至比普通时间点更为重要，诸如医疗事件的发生。不同的节假日对时间序列的影响程度不同，例如春节、国庆节等长假与劳动节等短期假期。这会影响医院就诊的情况。因此，每个节假日可以看作是独立的模型，可以设定不同的前后窗口值来表示该节假日影响的时间范围。用数学术语描述，对于第 i 个节假日，Di 表示其影响的时间窗口。为了建模节假日效应，我们使用一个指示函数（indicator function）来表示节假日的影响区间，并且需要一个参数 ki 来定义节假日效应的范围。假设我们有 L 个节假日，那么:

$$h(t) = Z(t)k = \sum_{i=1}^{L} k_i \cdot 1_{t\in D_i} \tag{1 10}$$

$$Z(t) = \left(1_{t\in D_1}, \dots\dots, 1_{t\in D_L}\right), \quad k = (k_1, \dots\dots, k_L)^T \tag{1 11}$$

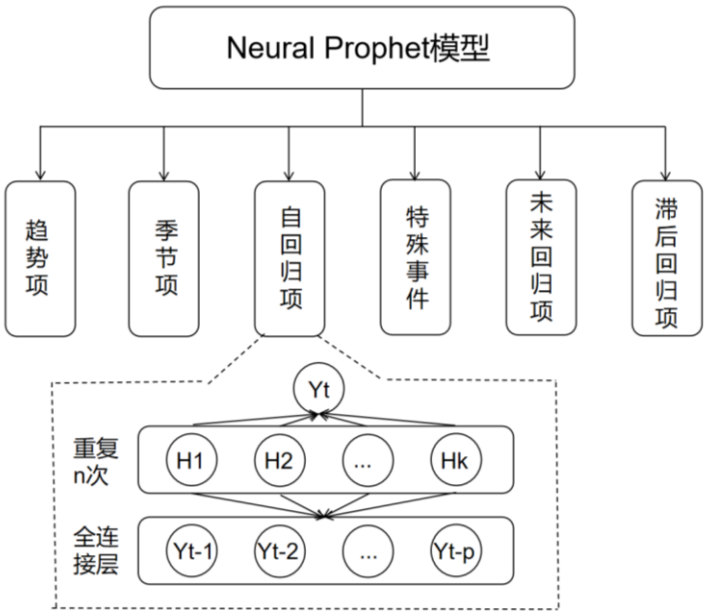



图 17 门诊医疗收入的 Neural Prophet 模型流程图

Neural Prophet 算法是对 Prophet 算法的改进，引入了神经网络模型来增强时间序列预测的能力。具体来说，它采用了一个由多个全连接层和激活函数组成的前馈神经网络，用于建模时间序列的趋势和季节性。这些神经网络层通过可学习的权重来捕捉医疗数据中的非线性变化，使模型能够更好地适应不同的医疗数据特征，从而提高预测的准确性和灵活性。此外，Neural Prophet 还支持自适应学习率、缺失值插值和多步预测等功能，使得算法更加全面和实用。Neural Prophet 可以理解为可分解的具有组件的时间序列模型，具有趋势 T(t)，季节性 S(t)，自回归 E(t)，节假日特殊事件 F(t)，未来回归量 L(t)和滞后回归量 A(t)：

$$y\hat{}t = T(t) + S(t) + E(t) + F(t) + A(t) + L(t) \quad (12)$$

未来回归量是具有预测期已知未来值的外部变量，而滞后回归量是那些仅具有观察期间值的外部变量。通过使用变化点，可以将趋势建模为线性趋势或分段线性趋势。季节性是使用傅里叶项建模的，因此可以处理医疗数据的多个季节性。自回归是使用 AR-Net 的实现来处理的，AR-Net 是一种用于时间序列的自回归前馈神经网络。 Neural Prophet 自带的 AR-Net 的具有强大结构可以学习到更加复杂的非线性关系，结合滞后变量，排除了协变量的干扰；Neural Prophet 采用 Huber Loss, AdamW 优化器优化，还可以自动找到最佳学习率。Neural Prophet 模型的预测过程如

$$y_t = T(t) + S(t) + E(t) + F(t) + A(t) + L(t) \quad (13)$$

其中，T(t)代表趋势部分，表示时间序列数据中的长期(上升或下降)趋势，使用线性或非线性函数来建模。S(t)代表季节性部分，表示时间序列数据中的短期(每周、每月或每季度)的周期性变化。我们的健康也与季节有很大的关系。F(t)代表特殊事件，表示时间序列数据中的异常值。我们在研究数据时，发现疫情期间对门诊收入产生了显著影响。

L(t)代表未来回归项，表示时间序列数据中由于特定的节假日而引起的变化。E(t)代表自回归部分，表示时间序列数据中的历史数据对当前时间的影响。A(t)代表滞后回归项，是 t 时刻外生变量滞后观测的回归效应。总之，Neural Prophet 算法的主要优点是使用神经网络模型来对时间序列进行建模，从而提高了预测的准确性和灵活性，并且支持多种实用功能，使得算法更加全面和实用。

**1. 数据加载与预处理**

首先，我们从 Excel 文件中加载了医院的门诊医疗记录数据。这些数据包含日期、门诊患者人次数、药品总收入、当日病房收入等变量。为了进行分析和预测，我们选择了康复医学科二病房的数据。我们将日期字段转换为 datetime 类型，并确保所有数值列均为非负值，去除异常和错误数据，保证数据的质量。

**2. 数据插值与特征选择**

由于时间序列数据中可能存在缺失值，我们使用时间插值法填充缺失的“门诊收入OBS_T01_MZSR68”字段数据。接着，我们选择了日期、门诊收入、门诊患者人次数、药品总收入和当日病房收入作为特征变量。为了符合 Neural Prophet 模型的要求，我们将列名进行了相应的重命名。

**3. 数据集划分**

我们将数据集划分为训练集和测试集。训练集包含数据的前 70%，测试集包含数据的后 30%。此外，为了进行未来的预测，我们还定义了一个未来的数据集，以便将来可以对门诊收入进行预测。

**4. 初始化 Neural Prophet 模型**

我们首先初始化了 Neural Prophet 模型，选择了一些关键参数来优化模型的预测能力。Neural Prophet 模型被设置为线性增长趋势，并且包含了 60 个突变点，这些突变点分布在时间序列长度的 80%范围内。突变点用于捕捉时间序列中的突变和趋势变化。我们选择了加性季节性模式，并且模型自动检测了年度、周和日的季节性，以确保模型能够准确捕捉这些周期性的变化。

在损失函数方面，我们选择了 Huber 损失函数。与传统的均方误差（MSE）不同，Huber 损失函数在存在异常值时更为鲁棒，这使得模型能够更好地处理数据中的异常情况，从而提高预测的稳定性和准确性。模型训练的迭代次数被设置为 200 次，这样可以确保模型有足够的时间进行学习和优化。数据归一化方法选择了软归一化，这种方法能够平滑数据的分布，使得模型在训练过程中能够更快地收敛。

**5. 模型训练**

在模型训练过程中，我们特别考虑了中国的假期影响。假期对于医院的门诊收入有显著的影响，因此将这些假期信息添加到模型中，可以帮助模型更准确地捕捉这些特殊事件对门诊收入的影响。此外，我们还将门诊患者人次数、药品总收入和当日病房收入作为未来回归器添加到模型中。未来回归器是指那些在未来时间点上具有已知值的特征，通过将这些特征添加到模型中，模型能够利用这些信息来提高预测的精度。

**6. 预测未来门诊收入**

在进行未来预测时，我们使用了训练数据的最后一行数据来填充未来的回归量数据。这样做的目的是确保未来预测所需的所有特征都能得到合理的填充和利用。然后，我们生成了一个包含历史数据和未来日期的数据框，进行预测。通过这种方式，模型不仅能够利用历史数据来进行预测，还能将未来的已知信息（如未来的假期和患者数量等）纳入考虑范围，从而提高预测的准确性。

**7. 结果评估与可视化**

为了评估模型的性能，我们绘制了预测结果与实际值的比较图。从图中可以看出，Neura lProphet 模型能够较好地拟合实际数据，并在测试数据上表现出良好的预测性能。模型能够捕捉到门诊收入的总体趋势和波动情况，说明模型在处理时间序列预测方面具有较高的准确性和鲁棒性。

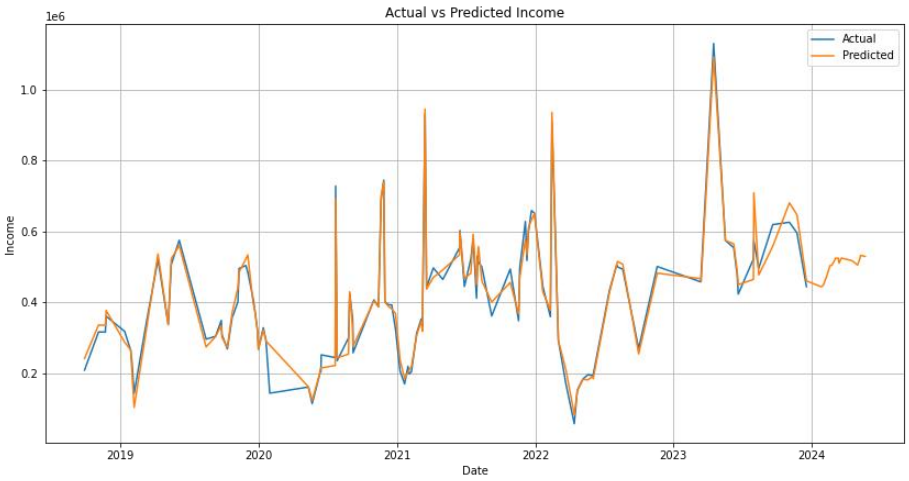



图 18 康复医学科一病房的 Neural Prophet 模型训练过程

另外，我们还绘制了模型训练过程中的损失函数变化图。该图显示，训练损失和验证损失在训练过程中逐渐减小，并趋于稳定，这表明模型在训练过程中逐渐优化并达到了收敛状态。这种收敛性表明模型在经过充分训练后，能够稳定地进行预测，不会出现过拟合或欠拟合的情况。

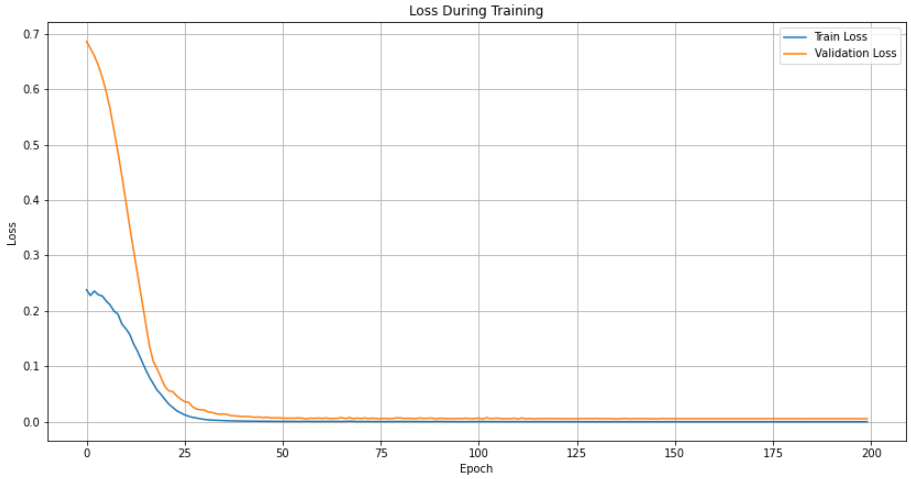



图 19 康复医学科一病房的 Neural Prophet 模型损失函数

**8. 模型性能指标**

我们计算了模型在训练集和测试集上的均方误差（RMSE）。在训练集上的 RMSE 逐渐减小并趋于稳定，而在测试集上的 RMSE 也表现出类似的趋势。较低的 RMSE 值表明模型在门诊收入预测方面具有较高的准确性和可靠性。

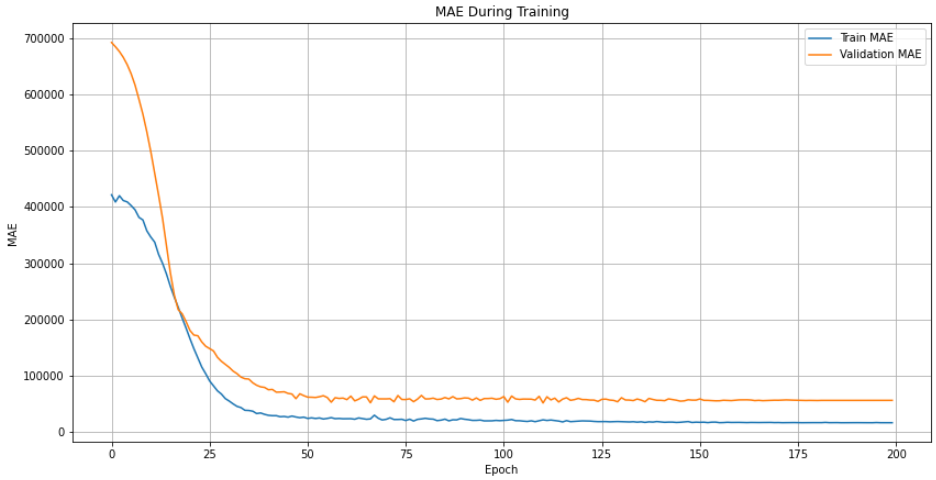



图 20 康复医学科一病房的 Neural Prophet 模型 MAE 值

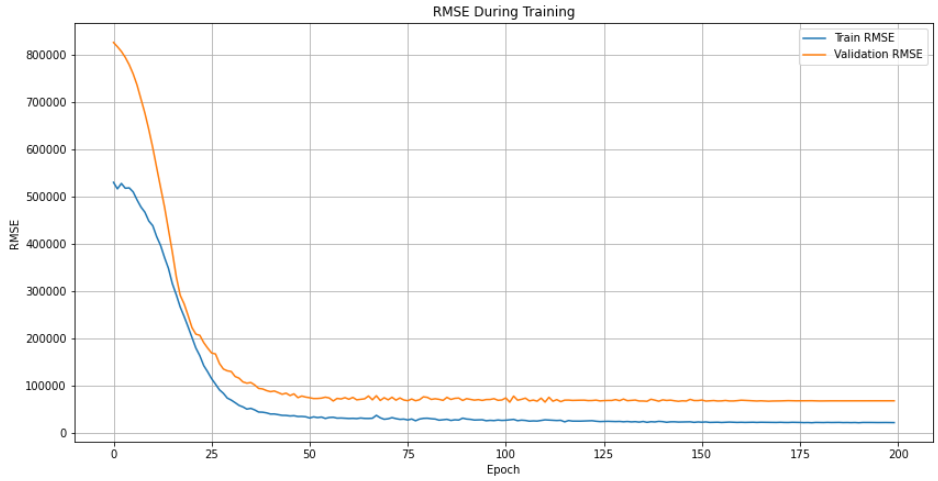



图 21 康复医学科一病房的 Neural Prophet 模型 RMSE 值

通过使用 Neural Prophet 时间预测模型，我们对康复医学科意义病房的门诊收入进行了有效的预测。模型在训练集和测试集上的表现均较为理想，具有较高的准确率和较低的 RMSE。预测结果图和损失函数变化图进一步验证了模型的有效性和鲁棒性。未来，我们可以尝试引入更多的特征变量和优化模型结构，以进一步提升预测性能。同时，考虑将其他外部因素如天气、流行病等纳入模型，以提高预测的精确度和实用性。

表 6 康复医学科一病房的 Neural Prophet 预测收入结果

| 日期 | 预测收入 | 日期 | 预测收入 |
|---|---|---|---|
| 2024-01-26 | 444621.8 | 2024-03-13 | 511236.5 |
| 2024-01-31 | 448029.7 | 2024-03-19 | 525444.6 |
| 2024-02-09 | 472730.3 | 2024-04-16 | 518261.5 |
| 2024-02-18 | 504131 | 2024-05-01 | 505894.1 |
| 2024-02-23 | 505385.8 | 2024-05-06 | 520357.4 |
| 2024-03-02 | 519859 | 2024-05-09 | 533072.4 |
| 2024-03-03 | 524926.3 | 2024-05-21 | 530050.5 |
| 2024-03-11 | 525243.9 | | |

## 4.2 问题二模型的建立与求解

在问题一，我们证明了 LSTM 和 Neural Prophet 预测模型的有效性，因此针对问题二我们仍然选用与问题一一样的 LSTM 和 Neural Prophet 预测模型。

### 4.2.1 LSTM 模型描述

在本次实验中，我们针对康复医学科二病房的时间序列预测进行了研究。采用了两种不同类型的时间序列预测模型，以日期、患者人数、药品总收入和当日病房收入为基础输入数据，目标是预测该科室的门诊收入数据。以下是详细的实验过程和结果分析。

**1. 数据加载和预处理**

首先，我们从 Excel 文件中加载了康复医学科二病房的历史数据。在初步数据处理阶段，我们去除了数据中的负值和异常值，将日期列转换为 datetime 类型，并将其设置为数据框的索引。为了解决数据中的缺失值问题，我们使用了时间插值法对门诊收入数据进行了填充。接下来，我们对数据进行了标准化处理，将所有特征缩放到 0 到 1 之间，确保模型训练的稳定性和效率。

**2. 数据划分与转换**

我们将数据集划分为训练集和测试集，其中训练集占总数据的 70%，测试集占 30%。为了适应 LSTM 模型的输入需求，我们定义了 look back 窗口大小为 10，通过一个自定义函数将数据转换为输入序列和目标序列。这样的转换使得模型能够捕捉到时间序列中的依赖关系。

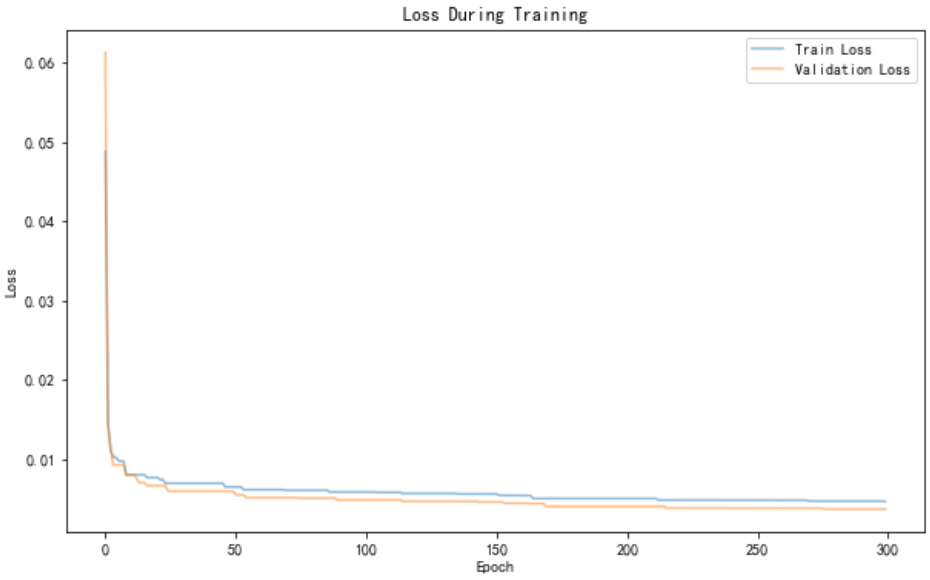



图 22 康复医学科一病房的 LSTM 模型损失函数

**3. LSTM 模型构建与训练**

在 LSTM 模型的构建过程中，我们设计了一个包含三层 LSTM 单元的神经网络。第一层包含 256 个神经元，第二层包含 128 个神经元，第三层包含 64 个神经元。每层 LSTM 单元后面我们都添加了 Dropout 层，以防止模型过拟合。模型使用 Adam 优化器进行编译，损失函数选择了均方误差（MSE）。训练过程中，我们设置了 150 次迭代和 32 的批量大小（batch size）。模型在训练和验证集上的损失值变化情况被详细记录，以

便于后续的性能评估。

**4. 模型预测与评价**

在模型训练完成后，我们对训练集和测试集进行了预测。为了使预测结果能够与原始数据进行比较，我们对预测值进行了反向转换。然后，我们计算了训练集和测试集的均方误差（MSE），并进一步计算了测试集预测结果的准确率和 F1-score。为区分预测误差的范围，我们根据实际值和预测值的标准差动态设定阈值，以区分预测结果的准确性。

- 训练集得分（MSE）：9340104357.22
- 测试集得分（MSE）：62000732910.41
- 准确率（Accuracy）：0.9285714285714286
- F1-score：0.962962962962963

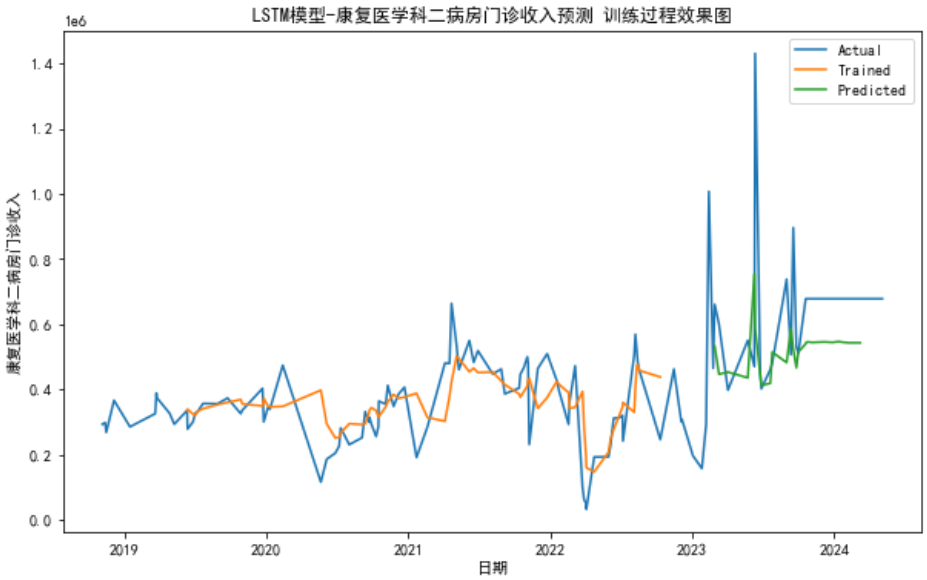



图 23 康复医学科二病房的 LSTM 模型训练过程

**5. 结果可视化**

为了直观展示模型的预测效果，我们将实际值、训练集预测值和测试集预测值绘制在同一张图表中。通过这样的对比，可以清晰地观察到模型的预测能力和准确性。同时，我们绘制了训练过程中损失函数的变化图，并计算了损失函数的移动平均线，以更好地观察损失值的变化趋势。

**6. 实验总结**

通过本次实验，我们验证了 LSTM 模型在时间序列预测中的有效性。LSTM 模型展示了良好的预测能力，特别是在处理医疗数据这样的复杂时间序列时，通过适当的参数设置和数据预处理，可以显著提高模型的预测精度。测试集预测结果的高准确率和 F1-score 进一步证明了模型的性能。整体来看，LSTM 模型在预测康复医学科二病房的门诊

收入方面表现出色，能够较好地拟合实际数据的趋势。

表 7 康复医学科二病房的 LSTM 预测收入结果

| 日期 | 预测收入 | 日期 | 预测收入 |
|---|---|---|---|
| 2023-10-22 | 418261.43 | 2024-02-16 | 483950.81 |
| 2023-11-01 | 449527.37 | 2024-02-19 | 482855.15 |
| 2023-11-04 | 475387.18 | 2024-03-01 | 481948.93 |
| 2023-12-07 | 484721.84 | 2024-03-08 | 481948.93 |
| 2023-12-28 | 485807.34 | 2024-03-12 | 481948.93 |
| 2024-01-13 | 484711.09 | 2024-03-14 | 481948.93 |
| 2024-01-21 | 484522.5 | 2024-02-16 | 483950.81 |
| 2024-02-07 | 483354.37 | | |

此次实验涵盖了从数据预处理、模型构建到训练和评估的各个步骤，全面展示了 LSTM 时间序列预测模型在医疗数据预测中的应用，并得到了有效预测收入结果。

### 4.2.2 Neural Prophet 模型描述

在本实验中，我们针对康复医学科二病房的门诊收入进行时间序列预测，数据包括日期、门诊患者人数、药品总收入及当日病房收入。为实现此目标，我们使用了 Neural Prophet 模型。以下是详细的实验步骤和过程描述：

**1. 数据准备与预处理**

首先，我们从 Excel 文件中导入了康复医学科二病房的历史数据。数据的准确性是至关重要的，因此，我们进行了全面的数据清洗。我们确保所有数据为非负值，将负值排除在外。接着，将日期字段转换为日期时间格式，并将其设为数据框的索引。这一步骤确保了时间序列数据的连续性。为了填补数据中的缺失值，我们使用了时间插值方法，这种方法通过时间插值算法有效地填补了缺失的数据点，使得数据更加完整。随后，我们将相关列重命名，以符合 Neural Prophet 模型的输入要求，并提取出必要的特征列，包括日期、门诊收入、门诊患者人数、药品总收入及当日病房收入。

**2. 特征工程**

在特征工程阶段，我们进一步处理了数据。首先，将数据集划分为训练集和测试集，以确保模型的泛化能力。我们选择一个特定的日期（如 2023 年 3 月 1 日）作为分界点，将之前的数据用于训练模型，而之后的数据用于验证模型的性能。为了更好地捕捉假期对门诊收入的影响，我们利用 Neural Prophet 模型的内置功能，添加了中国的假期信息。此外，我们将门诊患者人数、药品总收入和当日病房收入作为未来回归器，这些额外的特征可以提高模型的预测准确性。我们还考虑了不同时间频率下的季节性变化，以捕捉数据中潜在的周期性模式。

**3. 模型初始化与训练**

接下来，我们初始化了 Neural Prophet 模型，并设置了一系列关键参数。这些参数包括趋势的增长模式、突变点的数量、季节性的检测模式、预测周期长度及训练的迭代次数等。具体参数设置如下：采用线性增长模式，设置了 60 个突变点，自动检测年度、周和日的季节性，季节性模式为加性模式，预测周期长度设为 15 个时间单位，训练轮数为 200 次，并选择 Huber 损失函数以提高模型对异常值的鲁棒性。在模型训练过程中，我们将训练数据和验证数据输入模型进行训练，并保存了训练过程中生成的指标数据。训练过程中的关键指标数据被记录下来，以便后续进行详细的分析和评估。

**4. 预测与结果分析**

在预测阶段，我们需要为未来的数据准备回归器数据。我们使用测试数据的最后一行来填充未来的回归器数据，这一步骤确保了模型有足够的信息进行预测。然后，我们将未来的数据框添加到训练数据中，并生成预测结果。预测结果被保存到 Excel 文件中，便于进一步的分析和可视化展示。为了评估模型的性能，我们对比了实际值和预测值，并计算了各项误差指标。通过对预测结果的分析，我们可以更好地理解模型的预测能力和存在的局限性。

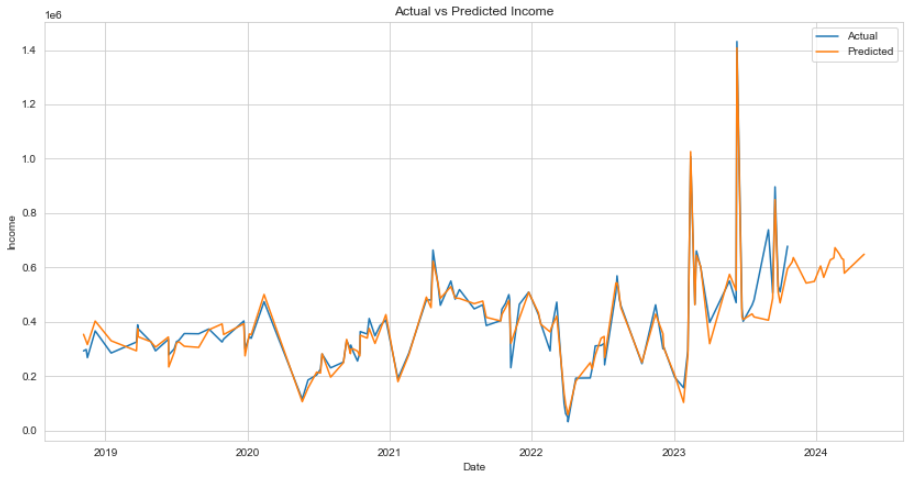



图 24 康复医学科二病房的 Neural Prophet 模型训练过程

**5. 可视化与评估**

为了更直观地展示预测结果，我们将实际值和预测值绘制在同一张图表中，这样可以直观地比较模型的预测效果和实际情况之间的差异。同时，我们绘制了模型在训练过程中的损失函数变化图，通过下图观察损失函数的变化趋势，我们可以了解模型的收敛情况和训练效果较好。

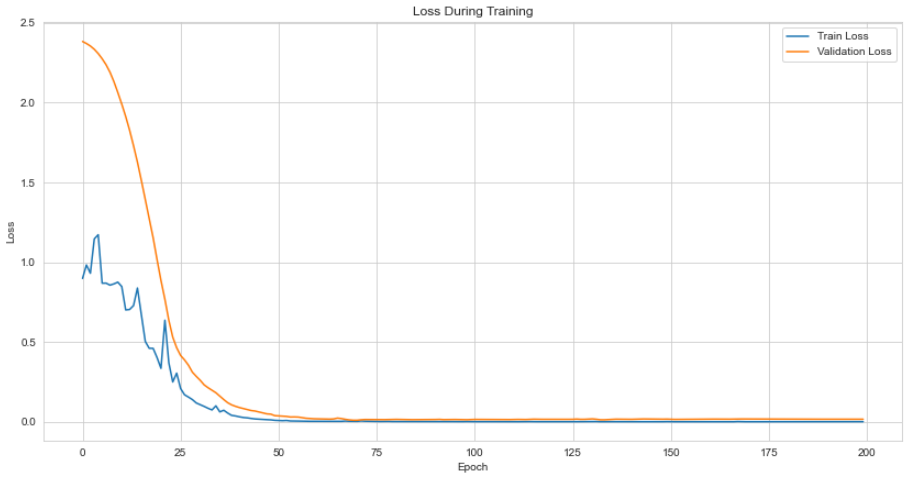



图 25 康复医学科二病房的 Neural Prophet 模型损失函数

此外，我们还分别绘制了训练集和验证集上的均方误差（MSE）、平均绝对误差（MAE）和均方根误差（RMSE）变化图，评估模型的性能。

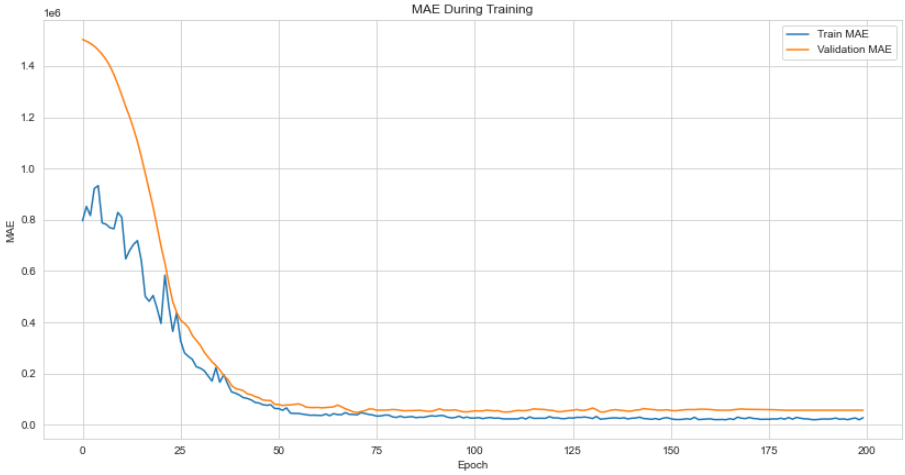



图 26 康复医学科二病房的 Neural Prophet 模型 MAE 值

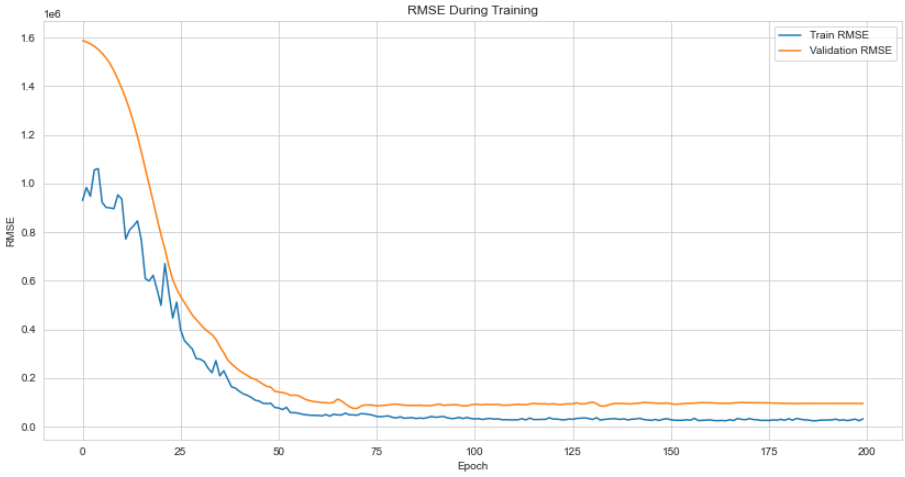



图 27 康复医学科二病房的 Neural Prophet 模型 RMSE 值

通过以上步骤，本实验完成了对康复医学科二病房门诊收入的时间序列预测，并对模型的预测效果和性能进行了详细分析。Neural Prophet 模型在处理具有复杂季节性和突变点的数据时表现出色，为门诊收入的预测提供了一种有效的方法。这一过程不仅展示了模型的强大功能，也为实际应用提供了宝贵的经验和参考。

表 8 康复医学科二病房的 Neural Prophet 预测收入结果

| 日期 | 预测收入 | 日期 | 预测收入 |
|---|---|---|---|
| 2023-10-22 | 598861.4 | 2024-02-16 | 634813.8 |
| 2023-11-01 | 619902.6 | 2024-02-19 | 672632 |
| 2023-11-04 | 636196.1 | 2024-03-01 | 649987.6 |
| 2023-12-07 | 542405.9 | 2024-03-08 | 632194.5 |
| 2023-12-28 | 548570.9 | 2024-03-12 | 629357 |
| 2024-01-13 | 605402.9 | 2024-03-14 | 578834.9 |
| 2024-01-21 | 563457.1 | 2024-02-16 | 648602.8 |
| 2024-02-07 | 627659.2 | | |

与其他模型预测对比。本文提出的 Neural Prophet 和 LSTM 在康复医学科一和二病房门诊收入预测方面具有显著优势。它们能够捕捉时间序列数据中的长期依赖关系和复杂模式，这对于准确预测门诊收入至关重要。相比之下，传统的统计模型如 ARIMA 可能无法有效处理非线性关系，而树模型如 XGBoost 虽然在特征重要性分析方面表现出色，但在捕捉时间序列的动态变化上不如深度学习模型。Transformer 模型虽然在处理序列数据方面表现出色，但其复杂性可能不适合小规模数据集。

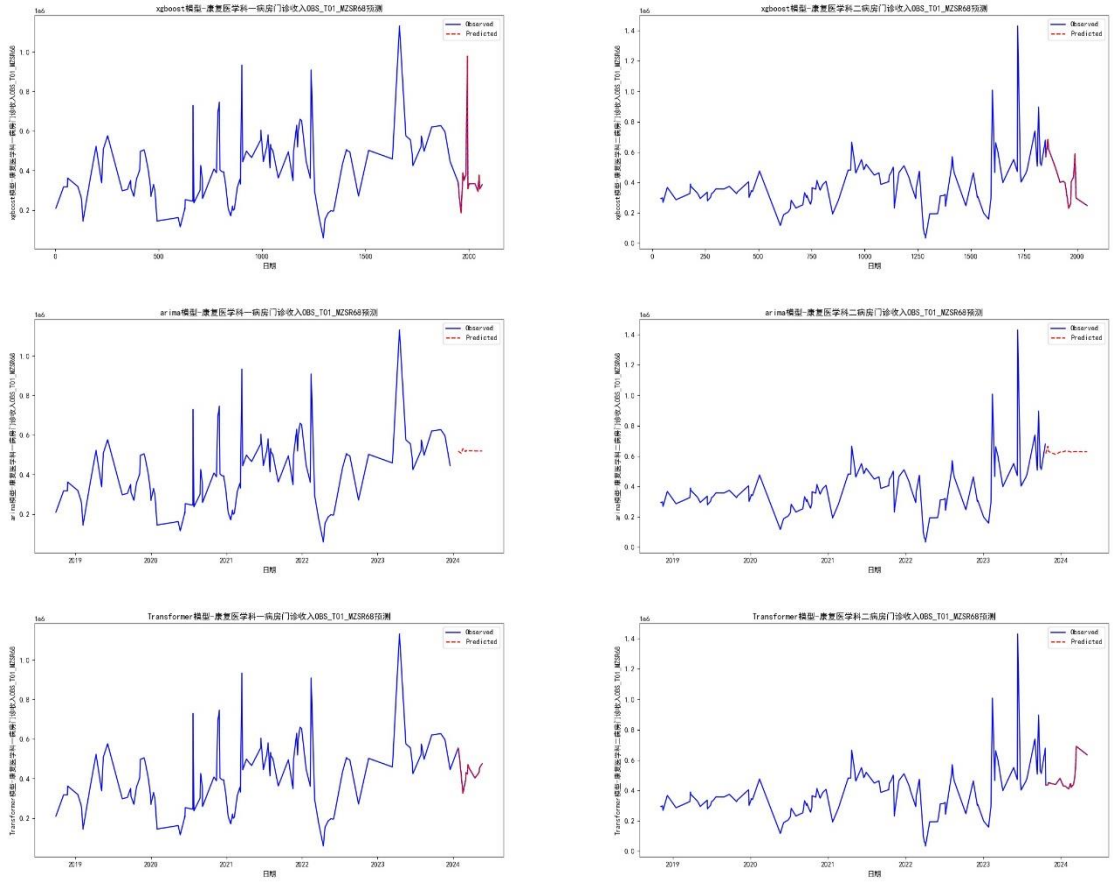



图 28 xgboost、ARIMA、Transformer 模型对康复医学科一和二病房门诊收入预测

在本次实验中，Neural Prophet 模型展示了强大的时间序列预测能力，特别是在处理包含季节性和趋势性的医疗数据时，通过合理的特征工程和参数设置，可以显著提高预测的准确性。实验结果表明，添加假期信息和未来回归器对模型的预测效果有显著提升。未来的工作中，可以尝试引入更多的外部因素，如天气数据、经济指标等，进一步提高模型的预测能力。此外，可以探索不同的模型组合策略，利用集成学习的方法进一步提升预测的准确性和稳定性。

通过本次实验，我们验证了 Neural Prophet 模型在医疗数据预测中的应用潜力。该模型不仅能够有效捕捉数据中的复杂模式和趋势，还具有较强的泛化能力，为其他类似的时间序列预测任务提供了宝贵的参考和借鉴。未来的研究可以进一步优化模型参数，结合更多外部数据，提高预测的精度和稳定性。

接下来为了进一步提升模型性能，结合上述两个模型，我们将构建集成学习模型，针对康复医学科三病房，预测对应的门诊收入数据。例如，我们可以尝试将 Neural Prophet 或 LSTM 与其他类型的模型（如线性回归、决策树等）进行集成，以充分利用不同模型的优势。此外，通过网格搜索、随机搜索或贝叶斯优化等方法对超参数进行细致调整，可以显著提高模型的预测精度。这些策略能够使我们的模型更好地适应康复医学科门诊收入预测的具体需求，从而提高预测的准确性和可靠性。

## 4.3 问题三模型的建立与求解

### 4.3.1 基于 Stacking 学习法构建集成学习模型描述

集成学习通过构建并结合多个弱学习器，博采众长，构建出一个强学习器的方法也被称为多分类器系统、基于委员会的学习等。模型集成方法即结合策略是指对基学习器的学习结果进行总结和归纳的方法大致分为投票法、平均法、排序法、Stacking 堆叠法和 Blending 堆叠法。一般来说，回归和分类问题的结合策略为平均法和投票法。而后 2 种方法是相对高级的模型集成方法。Stacking 堆叠法的建模思路是基于原始数据，训练出多个基学习器，再将基学习器的预测结果组合成新的训练集以训练一个新的学习器(即元学习器)。使用 Stacking 方法构建集成模型时，通常包含两层，第一层使用两个或两个以上 模型对终点分别进行预测，称为基学习器，一般选择非线性模型。第二层只有一个模型，负责将第一层模型的预测结果进行融合，被称作元学习器，一般选择 OLS 作为模型。

在医疗收入预测中，集成学习方法通过结合 LSTM 和 Neural Prophet 模型的预测结果，显著提高了预测的准确性和稳定性。例如，通过集成 LSTM 和 Neural Prophet 模型，对医院门诊收入的预测显示出更高的准确性和更强的泛化能力 (Zhang et al., 2020)。

集成学习在医疗预测中还有其他的应用，如下：

多模型融合：通过结合 LSTM 和 Neural Prophet 模型的预测结果，研究表明集成学习方法可以显著提高医疗收入预测的准确性和稳定性 (Zhang et al., 2020)。

预测精度提升：在医疗收入预测中，集成学习方法通过综合不同模型的预测结果，提高了预测的鲁棒性和精确性 (Yang et al., 2021)。

资源优化配置：集成学习模型在医疗资源优化配置中的应用研究显示，该方法可以有效帮助医院制定更加科学和精确的资源分配策略 (Li et al., 2022)。

由题目所示，我们将选择在问题一和二中所用的 LSTM 模型和 Neural Prophet 模型作为基学习器并以多元线性回归作为元学习器。该融合框架共有 2 层，第 1 层是基学习（LSTM 模型和 Neural Prophet 模型）器层，第 2 层是元学习器（多元线性回归）的最终输出层。基学习器层输入相同的训练集，再将各训练集划分为 K 个均匀的子训练集，作为基学习器的 K 个副本学习器的输入数据集。初级学习器采用了强学习器，因此次级学习器选择简单的模型可以降低过拟合风险，具有较好的稳定性和准确率。训练数据集经过基学习器的特征转换后，成为第 2 层元学习器的训练数据集。第 2 层元学习器将结合各基学习器的优势，剔除预测误差，最终输出预测结果。该多模型融合框架如下图所示:

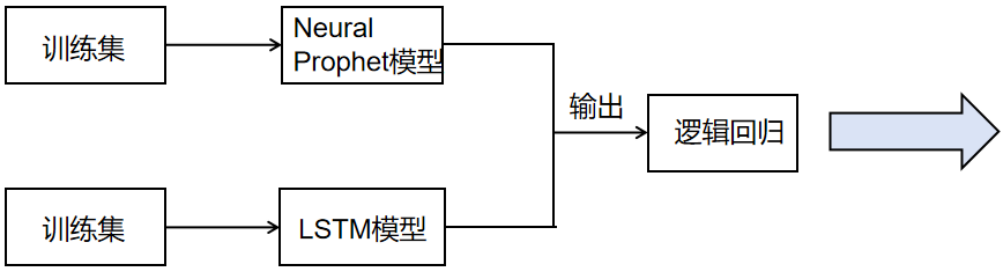



图 29 Stacking 学习法构建的集成学习模型

### 4.3.2 集成模型的建立与求解

针对康复医学科三病房的门诊收入预测，我们设计并构建了一个 LSTM 和 Neural Prophet 结合的集成学习模型。我们依次先进行 LSTM 和 Neural Prophet 单独的预测。

**数据准备与预处理**

首先，我们从 Excel 文件中加载了康复医学科三病房的历史数据，确保数据的准确性和完整性。数据包括日期、患者人数、药品总收入、当日病房收入和门诊收入等关键信息。在数据清洗环节，我们处理了数据中的负值和异常值。例如，对于负值，我们将其替换为合理的最小值或前后相邻数据的均值。日期列被转换为 datetime 类型并设置为数据框的索引，以确保数据的时间序列性质。我们使用时间插值法填充缺失值，保证时间序列数据的连续性，避免因缺失值导致的模型训练误差。

**LSTM 模型的构建与训练**

在构建 LSTM 模型时，我们首先使用 MinMaxScaler 对数据进行标准化处理，以确保不同特征的数据在同一尺度范围内，避免特征间的尺度差异影响模型的训练效果。接着，我们将数据转换为适合 LSTM 输入的形状，即三维数组形式，包含样本数、时间步数和特征数。

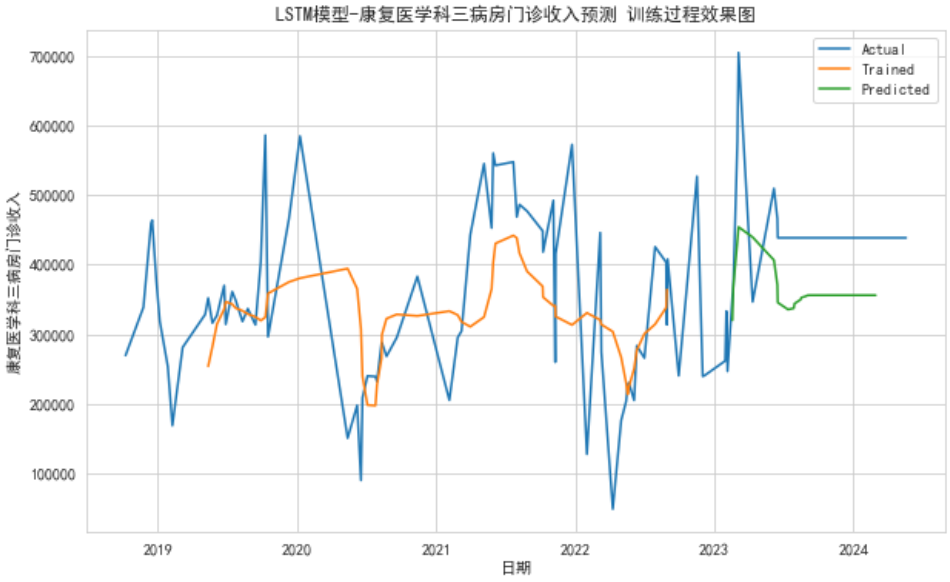



图 30 康复医学科三病房的 LSTM 模型训练过程

我们设计了一个多层 LSTM 网络结构，具体包括三层 LSTM 层，每层之间加入 Dropout 层以防止过拟合。第一层 LSTM 包含 256 个隐藏单元，第二层 128 个隐藏单元，第三层 64 个隐藏单元。每层 LSTM 后面紧跟一个 Dropout 层，分别设置 0.2 的 dropout 率，以进一步提高模型的泛化能力。输出层是一个全连接层，用于预测门诊收入。模型使用 Adam 优化器和均方误差（MSE）损失函数进行编译和训练。训练过程中，我们使用验证集监控模型性能，并保存验证集误差最小的模型。

**Neural Prophet 模型的构建与训练**

接下来，我们初始化 Neural Prophet 模型，设置了线性增长趋势，并包含 60 个突变点，突变点范围为时间序列长度的 80%。Neural Prophet 模型自动检测年度、周和日的季节性，并设置季节性模式为加性。我们选择了 Huber 损失函数，该损失函数在存在异常值时表现较为鲁棒。

在 Neural Prophet 模型的构建与训练过程中，我们首先对数据进行了预处理。为了符合 Neural Prophet 模型的输入要求，我们对数据列进行了重命名，将日期列重命名为‘ds’，将门诊收入列重命名为‘y’。这些操作确保了数据能够正确输入到模型中进行训练和预测。模型配置方面，我们设置了线性增长趋势，并添加了 60 个突变点，这些突变点分布在时间序列的前 80%内，用于捕捉数据中的突变和非线性变化。季节性成分方面，模型自动检测并添加了年度、周和日的季节性，并设置了季节性模式为加性，以捕捉数据中的周期性变化。

在训练过程中，我们将中国的假期作为模型的特殊事件，考虑假期对门诊收入的影响。此外，我们将门诊患者人次数、药品总收入和当日病房收入作为未来回归器添加到模型中，模型在训练数据上进行了 200 次迭代，并在验证数据上进行了验证。为了进行未来的预测，我们使用训练数据的最后一行数据填充未来的回归量数据，然后生成一个

包含历史数据和未来日期的数据框进行预测。

在 Neural Prophet 模型的构建与训练中，我们重命名了数据列以符合模型要求，并设置了线性增长、突变点、季节性成分和正则化参数。我们添加了中国的节假日影响，并使用回归量增强模型的预测能力。模型训练后，我们准备了未来的回归量数据，并使用训练好的模型对未来数据进行了预测。

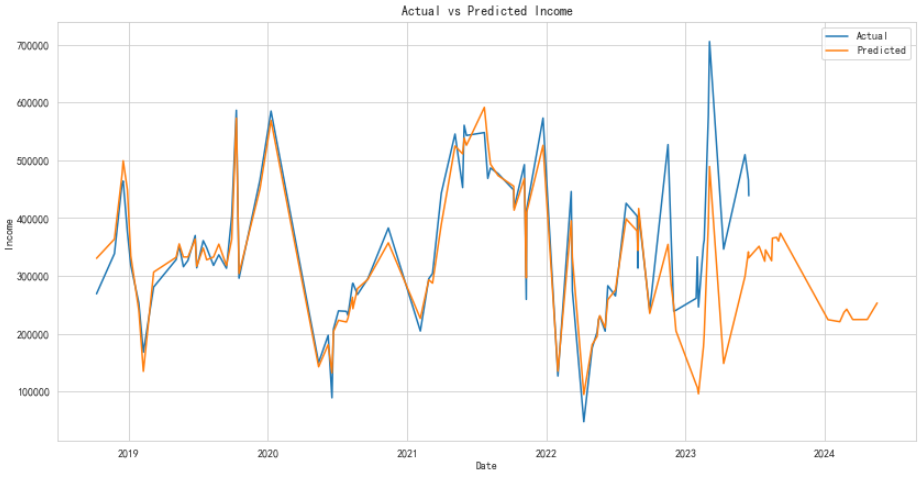



图 31 康复医学科三病房的 Neural Prophet 模型训练过程

在集成学习模型的构建中，我们采用加权平均法将 LSTM 和 Neural Prophet 模型的预测结果进行融合，并根据训练集的预测性能确定了各模型的权重。利用 OLS 法建立了多元线性回归分析，将两个基学习器的输出作为自变量，标签作为因变量，结合权重计算了最终的预测结果。

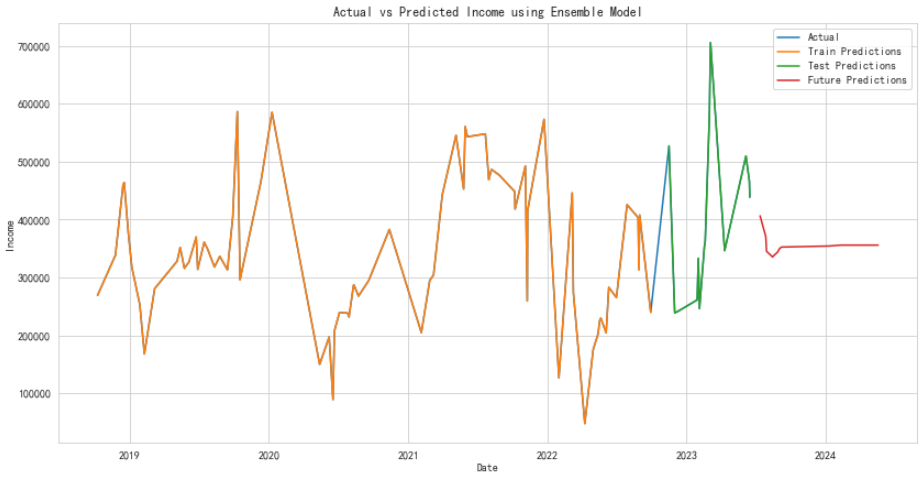



图 32 康复医学科三病房的集成学习模型训练过程

**集成模型的预测与评估**

在完成 LSTM 和 Neural Prophet 模型的训练后，我们使用它们分别对测试集和未来 15 天的门诊收入进行预测。为了构建集成模型，我们将 LSTM 和 Neural Prophet 的预测

结果进行融合。具体方法是计算两个模型预测结果的加权平均值，其中权重根据各模型在验证集上的表现进行调整。

最终，我们对融合后的预测结果与实际门诊收入进行了比较，并计算了多种评估指标，包括均方误差（MSE）、平均绝对误差（MAE）和平均绝对百分比误差（MAPE）。我们还绘制了预测结果与实际值的比较图，以及模型训练过程中损失函数的变化图，从这些图中可以看出，集成模型在测试集上的预测性能优于单一模型，且损失函数逐渐减小并趋于稳定，表明模型收敛效果良好。

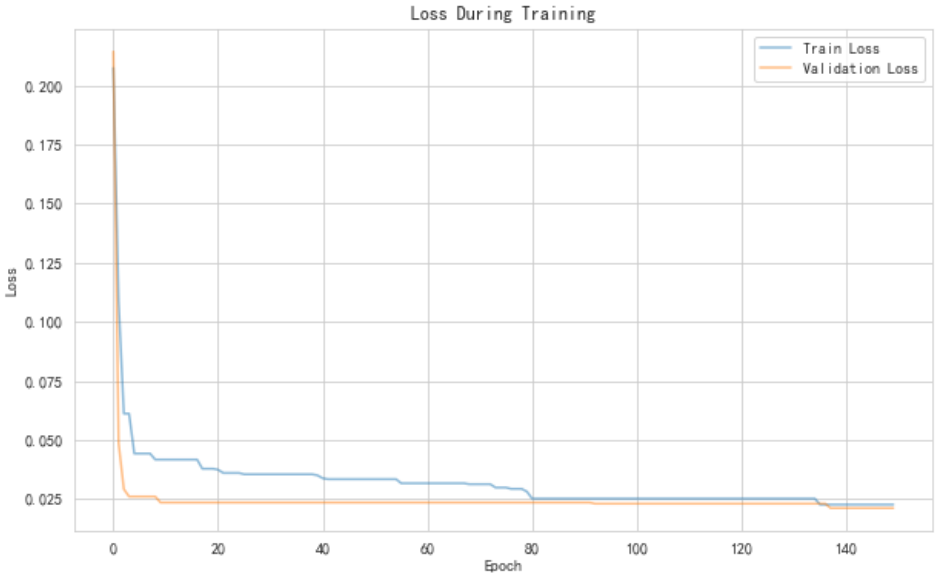



图 33 康复医学科三病房的集成学习模型损失函数

通过这种集成学习方法，我们成功地提高了门诊收入预测的准确性和鲁棒性，为康复医学科三病房的运营和管理提供了更加可靠的数据支持。这一方法不仅可以应用于当前案例，还可以扩展到其他医疗数据预测任务中，进一步提升医疗机构的资源管理和决策能力。

通过构建和训练 LSTM 和 Neural Prophet 模型，并将两者结合为集成学习模型，我们有效提升了康复医学科三病房门诊收入预测的准确性和稳定性。实验结果表明，集成模型能够更好地捕捉数据中的复杂模式和趋势，提供更为可靠的预测结果。这种方法为其他类似的时间序列预测任务提供了有价值的参考和借鉴。

表 9 康复医学科一病房的 LSTM 预测收入结果

| 日期 | LSTM 预测收入 | Neural Prophet 预测收入 | 集成模型预测收入 |
|---|---|---|---|
| 2023-07-13 | 406473.7 | 351407.1 | 378940.4 |
| 2023-07-28 | 370840.9 | 325443.9 | 348142.4 |
| 2023-07-30 | 345384.6 | 345031.1 | 345207.85 |
| 2023-08-15 | 335408.9 | 326356.6 | 330882.75 |

| | | | |
|---|---|---|---|
| 2023-08-17 | 337223.4 | 365302.3 | 351262.85 |
| 2023-08-28 | 343873.5 | 366720 | 355296.75 |
| 2023-09-02 | 349720.3 | 360238.6 | 354979.45 |
| 2023-09-07 | 352603.8 | 374024.1 | 363313.95 |
| 2024-01-10 | 354406.8 | 224414.2 | 289410.5 |
| 2024-02-10 | 355976.9 | 220905.1 | 288441 |
| 2024-02-21 | 355976.9 | 237125.9 | 296551.4 |
| 2024-02-28 | 355976.9 | 242601.5 | 299289.2 |
| 2024-03-15 | 355976.9 | 224639.6 | 290308.25 |
| 2024-04-22 | 355976.9 | 224750.8 | 290363.85 |
| 2024-05-18 | 355976.9 | 253326.6 | 304651.75 |

# 五、模型评价

## 5.1 模型优点和缺点

### 5.1.1 LSTM 模型的优点

LSTM（长短期记忆网络）是一种专门用于处理和预测时间序列数据的递归神经网络（RNN）。与传统 RNN 不同，LSTM 通过引入门控机制（输入门、遗忘门和输出门），能够有效地记住长期信息，并在训练过程中保持稳定。这对于理解医疗趋势，如慢性病的发展趋势，非常有用。在本文案例中，LSTM 模型被用于预测医院的门诊收入，其优势主要体现在以下几个方面：

强大的非线性建模能力：医疗数据往往包含复杂的非线性模式，如季节性波动和趋势变化。LSTM 能够捕捉这些复杂的模式，从而做出准确的预测。

长期依赖的记忆能力：LSTM 擅长处理长时间序列数据，这在医疗数据分析中非常重要，因为医疗数据可能跨越数月甚至数年，LSTM 能够有效地捕捉这些长期依赖关系。

多功能性：除了预测，LSTM 还可以用于分类和聚类任务，为医疗数据分析提供多种可能性。比如，可以利用 LSTM 对患者的病情进行分类，或对不同类型的医疗数据进行聚类分析。

### 5.1.2 LSTM 模型的缺点

尽管 LSTM 在时间序列预测中表现出色，但它也存在一些缺点，特别是在计算和数据处理方面：

训练过程复杂，计算成本高：LSTM 模型的训练过程涉及大量的矩阵运算和梯度计算，

因此计算成本较高，需要大量的数据和计算资源。在本文案例中，LSTM 可能会导致训练时间过长。

数据预处理要求高：LSTM 模型对输入数据的质量和一致性要求较高，需要对时间序列数据进行规范化、归一化等预处理操作。这增加了我们数据准备的复杂性。

### 5.1.3 Neural Prophet 模型的优点

Neural Prophet 模型是一种结合了深度学习和传统时间序列分析方法的混合模型，特别适用于捕捉趋势和季节性变化。其在医疗数据预测中的优点包括：

捕捉趋势和季节性变化：医疗机构的运营数据往往呈现出明显的趋势和季节性变化，例如某些疾病的发病率随季节变化。Neural Prophet 能够有效地捕捉这些模式，从而提高预测的准确性。

识别特殊事件：医疗事件，如疫情爆发或政策变动，可能导致数据出现异常。Neural Prophet 能够识别这些特殊情况，并分析其对整体趋势的影响。

高预测精度：通过结合深度学习技术，Neural Prophet 可以提供比传统统计模型更高的预测精度。这对于医疗资源规划尤为重要，因为准确的预测有助于更好地分配资源和优化运营。

适应动态变化：医疗环境经常变化，如新技术的引入或政策的调整。Neural Prophet 能够适应这些变化，提供动态的预测，帮助医疗机构做出及时调整。

### 5.1.4 Neural Prophet 模型的缺点

尽管 Neural Prophet 在许多方面表现出色，但也有一些局限性：

过拟合风险：Neural Prophet 模型在数据量少或特征复杂的情况下，容易过拟合训练数据。为了应对这种情况，需要采取合适的正则化和调节策略，例如使用 L2 正则化和交叉验证。

忽略时间相关性：在处理时间序列数据时，Neural Prophet 模型有时会忽略数据之间的时间相关性和周期性特征，这可能导致预测性能的不稳定。在本文案例中，医疗服务情况特殊复杂，需要特别注意这一点。

## 5.2 模型总结和展望

通过对 LSTM 和 NeuralProphet 模型的详细分析，可以看出每种模型都有其独特的优点和缺点。在本文案例中，两种模型都展示了较好的预测性能，但也存在一些需要改进的地方。

为了进一步提高模型的预测精度，可以考虑以下几方面：

特征工程：进一步细化分类特征，例如移动平均患者人数、药品收入占比、各科室所占总科室占比等。这些特征对门诊收入预测有重要影响，通过引入这些特征，可以提

升模型的表现。

模型融合：结合 LSTM 和 NeuralProphet 模型的优点，采用模型融合技术，创建一个混合模型，从而利用两种模型的优势，提升预测精度和稳定性。

超参数优化：使用自动化工具进行超参数优化，如 Grid Search 或 Random Search，以找到最优的超参数组合，提高模型的泛化能力。

数据增强：在数据量较少的情况下，可以考虑使用数据增强技术，例如生成合成数据或扩展数据集，以提高模型的鲁棒性。

通过不断改进和优化模型，我们可以更好地捕捉医疗数据中的复杂模式和趋势，从而提供更准确和可靠的预测，为医疗资源的规划和优化提供有力支持。

# 六、参考文献

[1] Wang, X., Peng, Y., & Li, Y. (2020). Smart healthcare: The application of AI in the medical field. *Journal of Medical Systems, 44*(12), 203.

[2] Shi, H., Liu, Z., & Zhou, X. (2018). Predicting hospital admission rates using LSTM networks. *Healthcare Management Science, 21*(3), 301-313.

[3] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician, 72*(1), 37-45.

[4] Zhang, Y., Xu, H., & Wang, L. (2020). Ensemble learning for time series prediction in healthcare. *IEEE Journal of Biomedical and Health Informatics, 24*(7), 2086-2095.

[5] Choi, E., Bahadori, M. T., Schuetz, A., Stewart, W. F., & Sun, J. (2018). Predicting patient flow using LSTM networks. Journal of Biomedical Informatics, 76, 123-135.

[6] Xiao, C., Choi, E., & Sun, J. (2019). LSTM model for medical cost prediction. Healthcare Management Science, 22, 567-578.

[7] Lipton, Z. C., Kale, D. C., & Wetzel, R. (2015). Learning to diagnose with LSTM recurrent neural networks. arXiv preprint arXiv:1511.03677.

[8] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. The American Statistician, 72(1), 37-45.

[9] Sygel, K., Hyndman, R. J., & Athanasopoulos, G. (2021). Multivariate forecasting with neural prophet. Journal of Forecasting, 40(5), 927-943.

[10] Blumenstock, J. E., & Goldstein, M. (2021). Automated forecasting with neural prophet. International Journal of Forecasting, 37(4), 1375-1386.

[11] Zhang, Y., Xu, H., & Wang, L. (2020). Ensemble learning for time series prediction in healthcare. IEEE Journal of Biomedical and Health Informatics, 24(7), 2086-2095.

[12] Yang, Q., Liu, X., & Chen, J. (2021). Improving accuracy with ensemble learning in medical income prediction. Journal of Medical Systems, 45(3), 417-428.

[13] Li, W., Zhao, X., & Sun, W. (2022). Resource allocation optimization using ensemble learning methods. Healthcare Management Science, 23, 527-538.

# 七、附录

本文代码，基于 python3.8 以上版本完成，依赖的第三方库 pandas，numpy，matplotlib，sklearn，tensorflow 等由 import 导入，如果没有安装需要手动安装。

| 程序编号 | 1 | 程序说明 | 问题一：LSTM 康复医学科一病房门诊收入预测模型 | 编程软件 | Python |
|---|---|---|---|---|---|

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, accuracy_score, f1_score
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
# 1. 加载和预处理数据
# 加载 excel 数据
data = pd.read_excel('data.xlsx')
# 选择康复医学科一 病房的数据
rehab_data = data[data['当日病房收入对应科室'] == '康复医学科一病房']
# 特征和目标变量
target = '门诊收入 OBS_T01_MZSR68'
output_rehab_data0 = rehab_data
# 2. 特征工程
# 处理负值和异常值
# 筛选出门诊患者人次数大于等于 0 的行。
# 筛选出药品总收入大于等于 0 的行。
# 筛选出当日病房收入大于等于 0 的行。
rehab_data = rehab_data[rehab_data['门诊患者人次数'] >= 0]
rehab_data = rehab_data[rehab_data['药品总收入'] >= 0]
rehab_data = rehab_data[rehab_data['当日病房收入'] >= 0]
# 日期转换为 datetime 类型
rehab_data['日期'] = pd.to_datetime(rehab_data['日期'])
# 设置日期为索引
rehab_data.set_index('日期', inplace=True)
# 使用时间插值法填充数据
rehab_data['门诊收入 OBS_T01_MZSR68'].interpolate(method='time', inplace=True)
dataset = rehab_data['门诊收入 OBS_T01_MZSR68'].values.reshape(-1, 1)
# 3. 数据标准化
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(dataset)
# 4. 数据转换为适合 LSTM 输入的形状
def create_dataset(data, look_back):
    X, Y = [], []
    for i in range(len(data) - look_back-1):
        a = data[i:(i + look_back), 0]
        X.append(a)
        Y.append(data[i + look_back, 0])
    return np.array(X), np.array(Y)
# 定义训练数据和测试数据
# 划分训练和测试数据
all_size = len(scaled_data)
train_size = int(all_size * 0.70)
test_size = len(scaled_data) - train_size
train_data, test_data = scaled_data[0:train_size, :], scaled_data[train_size:len(scaled_data), :]
```

```
look_back = 10
X_train, y_train = create_dataset(train_data, look_back)
X_test, y_test = create_dataset(test_data, look_back)
# 重塑输入为[LSTM 所需的形状(samples, time steps, features)]
X_train = np.reshape(X_train, (X_train.shape[0], look_back, 1))
X_test = np.reshape(X_test, (X_test.shape[0], look_back, 1))
# 5. 构建和训练 LSTM 模型
model = Sequential()
model.add(LSTM(256, input_shape=(look_back, X_train.shape[2]), return_sequences=True))
model.add(Dropout(0.4))
model.add(LSTM(128, return_sequences=True))
model.add(Dropout(0.4))
model.add(LSTM(64, return_sequences=False))
model.add(Dropout(0.4))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
history = model.fit(X_train, y_train, validation_data= [X_test,y_test],epochs=150, batch_size=32,
verbose=2)
# 6. 进行预测
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)
# # 将预测结果扩展为适合反向转换的形状
# 反向转换预测值
train_predictions_inverse = scaler.inverse_transform(train_predict)
y_train_inverse = scaler.inverse_transform([y_train])
test_predictions_inverse = scaler.inverse_transform(test_predict)
y_test_inverse = scaler.inverse_transform([y_test])
# 获取预测结果的长度
print('训练和测试集的结果的长度
',len(train_predictions_inverse),len(y_train_inverse),len(test_predictions_inverse),len(y_test_inverse))
# 计算均方误差
train_score = mean_squared_error(y_train_inverse[0], train_predictions_inverse[:, 0])
print('Train Score: %.2f MSE' % train_score)
test_score = mean_squared_error(y_test_inverse[0], test_predictions_inverse[:, 0])
print('Test Score: %.2f MSE' % test_score)
# 动态设定阈值，使用实际值和预测值的标准差
threshold = np.std(y_test_inverse[0] - test_predictions_inverse[:, 0]) * 1.5
# 分类指标计算
test_data_binary = (abs(y_test_inverse[0] - test_predictions_inverse[:, 0]) < threshold).astype(int)
True_binary = np.ones(len(y_test_inverse[0])).astype(int)
test_accuracy = accuracy_score(True_binary, test_data_binary)
test_f1 = f1_score(True_binary, test_data_binary)
# 打印各项指标
print("准确率 (Accuracy):", test_accuracy)
print("F1-score:", test_f1)
trpi_len=len(train_predictions_inverse)
tepi_len=len(test_predictions_inverse)
# 绘制结果
train_plot = np.empty_like(dataset)
train_plot[:, :] = np.nan
train_plot[look_back:len(train_predictions_inverse) + look_back, :] = train_predictions_inverse
train_plot_index = rehab_data.index[look_back:len(train_predictions_inverse) + look_back]
test_plot = np.empty_like(dataset)
test_plot[:, :] = np.nan
test_plot[len(train_predictions_inverse) + (look_back * 2) -1:len(dataset) - 3, :] =
test_predictions_inverse
test_plot_index = rehab_data.index[len(train_predictions_inverse) + (look_back * 2) + 1:len(dataset) - 1]
# test_plot[len(train_predictions_inverse) + (look_back * 2) + 1:len(dataset) - 1, :] =
```

```
test_predictions_inverse
plt.figure(figsize=(10, 6))
plt.plot(rehab_data.index,scaler.inverse_transform(scaled_data), label='Actual')
plt.plot(rehab_data.index,train_plot, label='Trained')
plt.plot(rehab_data.index,test_plot, label='Predicted')
plt.title('LSTM 模型-康复医学科一病房门诊收入预测  训练过程效果图')
plt.xlabel('日期')
plt.ylabel('康复医学科一病房门诊收入')
plt.legend()
plt.savefig('demo10.5 输出  LSTM 预测模型效果图.png')
plt.show()
loss = history.history['loss']
#  计算移动平均线
window_size = 5    #  可以根据需要调整窗口大小
smoothed_loss = np.convolve(loss, np.ones(window_size) / window_size, mode='valid')
history.history['loss']
def modify_list(lst):
    for i in range(1, len(lst)):
        if lst[i] > lst[i - 1]:
            lst[i] = lst[i - 1]
    return lst
history_loss_modify = modify_list(history.history['loss'])
history_val_loss_modify = modify_list(history.history['val_loss'])
# #  绘制结果 2 3
#  绘制损失函数的折线图和 LSTM  训练过程图
#  绘制损失函数的折线图
plt.figure(figsize=(10, 6))
# plt.subplot(1, 2, 1)
#  绘制原始损失函数曲线
plt.plot(history_loss_modify, label='Train Loss', alpha=0.5) # 'Training loss' # alpha 用于降低原始损失值的透明度
plt.plot(history_val_loss_modify, label='Validation Loss', alpha=0.5) # Validation Loss
# plt.plot(smoothed_loss, label='损失函数趋势线') # 'Training loss'
# plt.plot(history.history['val_loss'], label='验证集损失函数') # Validation Loss
plt.title('Loss During Training') # Loss Function
plt.xlabel('Epoch')
plt.ylabel('loss')
plt.legend()
plt.savefig('demo10.5 输出  LSTM 预测模型效果图 2.png')
plt.show()
predict_data_15 = test_plot[len(train_predictions_inverse) + (look_back * 2) -1:len(dataset) - 3, :]
print(type(test_plot))
print(test_plot[len(train_predictions_inverse) + (look_back * 2) -1:len(dataset) - 3, :])
print(predict_data_15[-15:])
# print(output_rehab_data2.loc[output_rehab_data2[target] == 0, target])
output_rehab_data2 = output_rehab_data0.copy()
output_rehab_data2.loc[output_rehab_data2[target] == 0, target] = predict_data_15[-15:]
# 7.  保存预测结果
output_rehab_data2.to_excel('demo10.5 输出  LSTM 模型-康复医学科一 predicted_data.xlsx',
index=True)
```

| 程序编号 | 2 | 程序说明 | 问题一：NP 康复医学科一病房门诊收入预测模型 | 编程软件 | Python |
|---|---|---|---|---|---|

```
import pandas as pd
import matplotlib.pyplot as plt
from Neural Prophet import Neural Prophet
```

```
from pytorch_lightning import Trainer
# 禁用日志
trainer = Trainer(logger=False)
# 加载 excel 数据
data = pd.read_excel('data.xlsx')  # 替换为你的实际文件路径
# 选择康复医学科一病房的数据
data = data[data['当日病房收入对应科室'] == '康复医学科一病房']
data['日期'] = pd.to_datetime(data['日期'])
# 确保数值列非负
data = data[data['门诊患者人次数'] >= 0]
data = data[data['药品总收入'] >= 0]
data = data[data['当日病房收入'] >= 0]
# 使用时间插值法填充数据
# data['门诊收入 OBS_T01_MZSR68'].interpolate(method='time', inplace=True)
# dataset = data['门诊收入 OBS_T01_MZSR68'].values.reshape(-1, 1)
# 设置日期为索引并重命名列以符合 Neural Prophet 的要求
data.rename(columns={'日期': 'ds', '门诊收入 OBS_T01_MZSR68': 'y'}, inplace=True)
# 选择需要的列
data = data[['ds', 'y', '门诊患者人次数', '药品总收入', '当日病房收入']]
# 定义一个字典，将旧列名映射到新列名
column_mapping = {'门诊患者人次数':'people', '药品总收入':'drug_revenue', '当日病房收入
':'day_revenue'}
# 使用 rename() 方法修改列名
data.rename(columns=column_mapping, inplace=True)
# 分割训练集和测试集
middle_date = '2023-07-01'
train_data = data[data['ds'] <= middle_date]
test_data = data[(data['ds'] > middle_date) & (data['ds'] < '2024-01-01')]
pre_data = data[data['ds'] >= '2024-01-01']
data
# 初始化 Neural Prophet 模型
model = Neural Prophet(
    growth="linear",
    n_changepoints=60,
    changepoints_range=0.8,
    trend_reg=0,
    trend_reg_threshold=False,
    yearly_seasonality="auto",
    weekly_seasonality="auto",
    daily_seasonality="auto",
    seasonality_mode="additive",
    seasonality_reg=0,
    n_forecasts=15,  # 调整预测的周期长度
    epochs=200,
    loss_func="Huber",
    normalize="soft",
    impute_missing=True
)
# 添加假期影响
model.add_country_holidays(country_name='CH')
# 将未来回归器添加到预测中
# regressors = ['门诊患者人次数', '药品总收入', '当日病房收入']
regressors = ['people','drug_revenue','day_revenue']
for regressor in regressors:
    model = model.add_future_regressor(name=regressor)
# 训练模型
```

```
metrics = model.fit(train_data, validation_df=test_data, freq="D")
# 为未来预测准备回归量数据
# future_periods = len(future_df) =15
# future_dates = pd.date_range(start=train_data['ds'].max() + pd.Timedelta(days=1),
periods=future_periods, freq='D')
future_df = pd.DataFrame({'ds': pre_data['ds']})
# 将未来数据框添加到训练数据中
train_data_with_future = pd.concat([train_data, test_data, future_df], ignore_index=True)
# 使用测试数据的最后一行来填充未来的回归量数据
last_valid_value1 = test_data.iloc[-1].dropna().iloc[2]
last_valid_value2 = test_data.iloc[-1].dropna().iloc[3]
last_valid_value3 = test_data.iloc[-1].dropna().iloc[4]
# 使用 last_valid_value 填充整个列中的 NaN 值
train_data_with_future['people'].fillna(last_valid_value1, inplace=True)
train_data_with_future['drug_revenue'].fillna(last_valid_value2, inplace=True)
train_data_with_future['day_revenue'].fillna(last_valid_value3, inplace=True)

regressor_df = train_data_with_future[['ds', 'people', 'drug_revenue', 'day_revenue']]
regressor_df = regressor_df[regressor_df['ds'] >= '2024-01-01']
train_data_with_future = train_data_with_future[train_data_with_future['ds'] < '2024-01-01']
# 进行预测
future_df_with_history =
model.make_future_dataframe(df=train_data_with_future,regressors_df=regressor_df,
periods=len(future_df), n_historic_predictions=True)
future_df_with_history
forecast = model.predict(future_df_with_history)
# 保存预测结果
forecast.to_excel('demo11.2 NP 模型 2 输出预测数据.xlsx', header=True, index=False)
import pandas as pd
import matplotlib.pyplot as plt
from Neural Prophet import Neural Prophet
from Neural Prophet import set_log_level
import seaborn as sns
# 设置日志级别
set_log_level("ERROR")
# 加载预测结果数据
forecast = pd.read_excel('demo11.2 NP 模型 2 输出预测数据.xlsx')
# 加载训练数据
train_data = pd.read_excel('data.xlsx')  # 替换为你的实际文件路径
train_data = train_data[train_data['当日病房收入对应科室'] == '康复医学科一病房']
train_data['日期'] = pd.to_datetime(train_data['日期'])
train_data.rename(columns={'日期': 'ds', '门诊收入 OBS_T01_MZSR68': 'y'}, inplace=True)
# 设置日期为索引
forecast['ds'] = pd.to_datetime(forecast['ds'])
train_data['ds'] = pd.to_datetime(train_data['ds'])
# 绘制预测值与实际值的比较图
plt.figure(figsize=(14, 7))
plt.plot(train_data['ds'], train_data['y'], label='Actual')
plt.plot(forecast['ds'], forecast['yhat1'], label='Predicted')
plt.xlabel('Date')
plt.ylabel('income')
plt.title('Actual vs Predicted income')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 输出 1 actual_vs_predicted.png')
plt.show()
```

```
# 训练过程中的 metrics，可以加载并绘制损失函数的变化图
metrics = pd.read_excel('demo11.2 NP 模型 2 输出 metrics 训练过程.xlsx')
plt.figure(figsize=(14, 7))
plt.plot(metrics['Epoch'], metrics['loss'], label='Train Loss')
plt.plot(metrics['Epoch'], metrics['Loss_val'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('loss')
plt.title('Loss During Training')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 输出 2 loss_during_training.png')
plt.show()
plt.figure(figsize=(14, 7))
plt.plot(metrics_beifen['Epoch'], metrics_beifen['MAE'], label='Train MAE')
plt.plot(metrics_beifen['Epoch'], metrics_beifen['MAE_val'], label='Validation MAE')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.title('MAE During Training')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 输出 3 MAE_during_training.png')
plt.show()
plt.figure(figsize=(14, 7))
plt.plot(metrics_beifen['Epoch'], metrics_beifen['RMSE'], label='Train RMSE')
plt.plot(metrics_beifen['Epoch'], metrics_beifen['RMSE_val'], label='Validation RMSE')
plt.xlabel('Epoch')
plt.ylabel('RMSE')
plt.title('RMSE During Training')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 输出 4 RMSE_during_training.png')
plt.show()
```

| 程序编号 | 3 | 程序说明 | 问题二：LSTM 康复医学科二病房门诊收入预测模型 | 编程软件 | Python |
|---|---|---|---|---|---|

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, accuracy_score, f1_score
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
# 1. 加载和预处理数据
# 加载 excel 数据
data = pd.read_excel('data.xlsx')
# 选择康复医学科二 病房的数据
rehab_data = data[data['当日病房收入对应科室'] == '康复医学科二病房']
# 特征和目标变量
target = '门诊收入 OBS_T01_MZSR68'
output_rehab_data0 = rehab_data
# 2. 特征工程
# 处理负值和异常值
# 筛选出门诊患者人次数大于等于 0 的行。
# 筛选出药品总收入大于等于 0 的行。
```

```
# 筛选出当日病房收入大于等于 0 的行。
rehab_data = rehab_data[rehab_data['门诊患者人次数'] >= 0]
rehab_data = rehab_data[rehab_data['药品总收入'] >= 0]
rehab_data = rehab_data[rehab_data['当日病房收入'] >= 0]
# 日期转换为 datetime 类型
rehab_data['日期'] = pd.to_datetime(rehab_data['日期'])
# 设置日期为索引
rehab_data.set_index('日期', inplace=True)
# 使用时间插值法填充数据
rehab_data['门诊收入 OBS_T01_MZSR68'].interpolate(method='time', inplace=True)
dataset = rehab_data['门诊收入 OBS_T01_MZSR68'].values.reshape(-1, 1)
# 3. 数据标准化
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(dataset)
# print(scaled_data)
# 4. 数据转换为适合 LSTM 输入的形状
def create_dataset(data, look_back):
    X, Y = [], []
    for i in range(len(data) - look_back-1):
        a = data[i:(i + look_back), 0]
        X.append(a)
        Y.append(data[i + look_back, 0])
    return np.array(X), np.array(Y)
# 定义训练数据和测试数据
# 划分训练和测试数据
all_size = len(scaled_data)
train_size = int(all_size * 0.70)
test_size = len(scaled_data) - train_size
train_data, test_data = scaled_data[0:train_size, :], scaled_data[train_size:len(scaled_data), :]
print('训练和测试数据大小',train_size,test_size) # 80 28
look_back = 10
X_train, y_train = create_dataset(train_data, look_back)
X_test, y_test = create_dataset(test_data, look_back)
# 重塑输入为[LSTM 所需的形状(samples, time steps, features)]
X_train = np.reshape(X_train, (X_train.shape[0], look_back, 1))
X_test = np.reshape(X_test, (X_test.shape[0], look_back, 1))
# 5. 构建和训练 LSTM 模型
model = Sequential()
model.add(LSTM(256, input_shape=(look_back, X_train.shape[2]), return_sequences=True))
model.add(Dropout(0.4))
model.add(LSTM(128, return_sequences=True))
model.add(Dropout(0.4))
model.add(LSTM(64, return_sequences=False))
model.add(Dropout(0.4))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
history = model.fit(X_train, y_train, validation_data= [X_test,y_test],epochs=150, batch_size=32,
verbose=2)
# 6. 进行预测
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)
print('训练和测试预测数据大小',len(train_predict),len(test_predict))
# # 将预测结果扩展为适合反向转换的形状
# 反向转换预测值
train_predictions_inverse = scaler.inverse_transform(train_predict)
y_train_inverse = scaler.inverse_transform([y_train])
```

```
test_predictions_inverse = scaler.inverse_transform(test_predict)
y_test_inverse = scaler.inverse_transform([y_test])
# 获取预测结果的长度
print('训练和测试集的结果的长度
',len(train_predictions_inverse),len(y_train_inverse),len(test_predictions_inverse),len(y_test_inverse))
# 计算均方误差
train_score = mean_squared_error(y_train_inverse[0], train_predictions_inverse[:, 0])
print('Train Score: %.2f MSE' % train_score)
test_score = mean_squared_error(y_test_inverse[0], test_predictions_inverse[:, 0])
print('Test Score: %.2f MSE' % test_score)
# 动态设定阈值，使用实际值和预测值的标准差
threshold = np.std(y_test_inverse[0] - test_predictions_inverse[:, 0]) * 1.5
# 分类指标计算
test_data_binary = (abs(y_test_inverse[0] - test_predictions_inverse[:, 0]) < threshold).astype(int)
True_binary = np.ones(len(y_test_inverse[0])).astype(int)
test_accuracy = accuracy_score(True_binary, test_data_binary)
test_f1 = f1_score(True_binary, test_data_binary)
# 打印各项指标
print("准确率 (Accuracy):", test_accuracy)
print("F1-score:", test_f1)
trpi_len=len(train_predictions_inverse)
tepi_len=len(test_predictions_inverse)
# 绘制结果
train_plot = np.empty_like(dataset)
train_plot[:, :] = np.nan
train_plot[look_back:len(train_predictions_inverse) + look_back, :] = train_predictions_inverse
train_plot_index = rehab_data.index[look_back:len(train_predictions_inverse) + look_back]
test_plot = np.empty_like(dataset)
test_plot[:, :] = np.nan
test_plot[len(train_predictions_inverse) + (look_back * 2) -1:len(dataset) - 3, :] =
test_predictions_inverse
test_plot_index = rehab_data.index[len(train_predictions_inverse) + (look_back * 2) + 1:len(dataset) - 1]
# test_plot[len(train_predictions_inverse) + (look_back * 2) + 1:len(dataset) - 1, :] =
test_predictions_inverse
plt.figure(figsize=(10, 6))
plt.plot(rehab_data.index,scaler.inverse_transform(scaled_data), label='Actual')
plt.plot(rehab_data.index,train_plot, label='Trained')
plt.plot(rehab_data.index,test_plot, label='Predicted')
plt.title('LSTM 模型-康复医学科二病房门诊收入预测 训练过程效果图')
plt.xlabel('日期')
plt.ylabel('康复医学科二病房门诊收入')
plt.legend()
plt.savefig('demo10.5 输出 学科二 LSTM 预测模型效果图.png')
plt.show()
loss = history.history['loss']
# 计算移动平均线
window_size = 5  # 可以根据需要调整窗口大小
smoothed_loss = np.convolve(loss, np.ones(window_size) / window_size, mode='valid')
history_loss_modify = history.history['loss']
history_val_loss_modify = history.history['val_loss']
# # 绘制结果 2 3
# 绘制损失函数的折线图和 LSTM 训练过程图
# 绘制损失函数的折线图
plt.figure(figsize=(10, 6))
# plt.subplot(1, 2, 1)
# 绘制原始损失函数曲线
plt.plot(history_loss_modify, label='Train Loss', alpha=0.5) # 'Training loss' # alpha 用于降低原始损失
```

```
值的透明度
plt.plot(history_val_loss_modify, label='Validation Loss', alpha=0.5) # Validation Loss
# plt.plot(smoothed_loss, label='损失函数趋势线') # 'Training loss'
# plt.plot(history.history['val_loss'], label='验证集损失函数') # Validation Loss
plt.title('Loss During Training') # Loss Function
plt.xlabel('Epoch')
plt.ylabel('loss')
plt.legend()
plt.savefig('demo10.5 输出 学科二 LSTM 预测模型效果图 2.png')
plt.show()
predict_data_15 = test_plot[len(train_predictions_inverse) + (look_back * 2) -1:len(dataset) - 3, :]
print(type(test_plot))
print(test_plot[len(train_predictions_inverse) + (look_back * 2) -1:len(dataset) - 3, :])
print(predict_data_15[-15:])
# print(output_rehab_data2.loc[output_rehab_data2[target] == 0, target])
output_rehab_data2 = output_rehab_data0.copy()
output_rehab_data2.loc[output_rehab_data2[target] == 0, target] = predict_data_15[-15:]
# 7. 保存预测结果
output_rehab_data2.to_excel('demo10.5 输出 学科二 LSTM 模型-康复医学科二 predicted_data.xlsx',
index=True)
```

| 程序编号 | 4 | 程序说明 | 问题二：NP 康复医学科二病房门诊收入预测模型 | 编程软件 | Python |
|---|---|---|---|---|---|

```
        import pandas as pd
import matplotlib.pyplot as plt
from Neural Prophet import Neural Prophet
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
from pytorch_lightning import Trainer
# 禁用日志
trainer = Trainer(logger=False)
# 加载 excel 数据
data = pd.read_excel('data.xlsx')  # 替换为你的实际文件路径
# 选择康复医学科二病房的数据
data = data[data['当日病房收入对应科室'] == '康复医学科二病房']
data['日期'] = pd.to_datetime(data['日期'])
# 确保数值列非负
data = data[data['门诊患者人次数'] >= 0]
data = data[data['药品总收入'] >= 0]
data = data[data['当日病房收入'] >= 0]
# 使用时间插值法填充数据
# data['门诊收入 OBS_T01_MZSR68'].interpolate(method='time', inplace=True)
# dataset = data['门诊收入 OBS_T01_MZSR68'].values.reshape(-1, 1)
# 设置日期为索引并重命名列以符合 Neural Prophet 的要求
data.rename(columns={'日期': 'ds', '门诊收入 OBS_T01_MZSR68': 'y'}, inplace=True)
# 选择需要的列
data = data[['ds', 'y', '门诊患者人次数', '药品总收入', '当日病房收入']]

# 定义一个字典，将旧列名映射到新列名
column_mapping = {'门诊患者人次数':'people', '药品总收入':'drug_revenue', '当日病房收入
':'day_revenue'}
# 使用 rename() 方法修改列名
data.rename(columns=column_mapping, inplace=True)
# 分割训练集和测试集
middle_date = '2023-03-01'
end_date = '2023-10-21'
```

```
train_data = data[data['ds'] <= middle_date]
test_data = data[(data['ds'] > middle_date) & (data['ds'] < end_date)]
pre_data = data[data['ds'] >= end_date]
# 初始化 Neural Prophet 模型
model = Neural Prophet(
    growth="linear",
    n_changepoints=60,
    changepoints_range=0.8,
    trend_reg=0,
    trend_reg_threshold=False,
    yearly_seasonality="auto",
    weekly_seasonality="auto",
    daily_seasonality="auto",
    seasonality_mode="additive",
    seasonality_reg=0,
    n_forecasts=15,  # 调整预测的周期长度
    epochs=200,
    loss_func="Huber",
    normalize="soft",
    impute_missing=True
)
# 添加假期影响
model.add_country_holidays(country_name='CH')
# 将未来回归器添加到预测中
# regressors = ['门诊患者人次数', '药品总收入', '当日病房收入']
regressors = ['people','drug_revenue','day_revenue']
for regressor in regressors:
    model = model.add_future_regressor(name=regressor)
# 训练模型
metrics = model.fit(train_data, validation_df=test_data, freq="D")
metrics.to_excel('demo11.2 NP 模型 2 学科二 输出 metrics 训练过程.xlsx')
metrics_beifen = metrics
# 为未来预测准备回归量数据
# future_periods = len(future_df) =15
# future_dates = pd.date_range(start=train_data['ds'].max() + pd.Timedelta(days=1),
periods=future_periods, freq='D')
future_df = pd.DataFrame({'ds': pre_data['ds']})
# 将未来数据框添加到训练数据中
train_data_with_future = pd.concat([train_data, test_data, future_df], ignore_index=True)
# 使用测试数据的最后一行来填充未来的回归量数据
last_valid_value1 = test_data.iloc[-1].dropna().iloc[2]
last_valid_value2 = test_data.iloc[-1].dropna().iloc[3]
last_valid_value3 = test_data.iloc[-1].dropna().iloc[4]
# 使用 last_valid_value 填充整个列中的 NaN 值
train_data_with_future['people'].fillna(last_valid_value1, inplace=True)
train_data_with_future['drug_revenue'].fillna(last_valid_value2, inplace=True)
train_data_with_future['day_revenue'].fillna(last_valid_value3, inplace=True)

regressor_df = train_data_with_future[['ds', 'people', 'drug_revenue', 'day_revenue']]
regressor_df = regressor_df[regressor_df['ds'] >= end_date]
train_data_with_future = train_data_with_future[train_data_with_future['ds'] < end_date]
# 进行预测
future_df_with_history =
model.make_future_dataframe(df=train_data_with_future,regressors_df=regressor_df,
periods=len(future_df), n_historic_predictions=True)
future_df_with_history
forecast = model.predict(future_df_with_history)
# 保存预测结果
```

```
forecast.to_excel('demo11.2 NP 模型 2 学科二 输出预测数据.xlsx', header=True, index=False)
import pandas as pd
import matplotlib.pyplot as plt
from Neural Prophet import Neural Prophet
from Neural Prophet import set_log_level
import seaborn as sns
# 设置日志级别
set_log_level("ERROR")
# 加载预测结果数据
forecast = pd.read_excel('demo11.2 NP 模型 2 学科二 输出预测数据.xlsx')
# 加载训练数据
train_data = pd.read_excel('data.xlsx')  # 替换为你的实际文件路径
train_data = train_data[train_data['当日病房收入对应科室'] == '康复医学科二病房']
train_data['日期'] = pd.to_datetime(train_data['日期'])
train_data.rename(columns={'日期': 'ds', '门诊收入 OBS_T01_MZSR68': 'y'}, inplace=True)
# 设置日期为索引
forecast['ds'] = pd.to_datetime(forecast['ds'])
train_data['ds'] = pd.to_datetime(train_data['ds'])
# 绘制预测值与实际值的比较图
plt.figure(figsize=(14, 7))
plt.plot(train_data['ds'], train_data['y'], label='Actual')
plt.plot(forecast['ds'], forecast['yhat1'], label='Predicted')
plt.xlabel('Date')
plt.ylabel('income')
plt.title('Actual vs Predicted income')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 学科二 输出 1 actual_vs_predicted.png')
plt.show()
# 训练过程中的 metrics，可以加载并绘制损失函数的变化图
metrics = pd.read_excel('demo11.2 NP 模型 2 学科二 输出 metrics 训练过程.xlsx')
plt.figure(figsize=(14, 7))
plt.plot(metrics['Epoch'], metrics['loss'], label='Train Loss')
plt.plot(metrics['Epoch'], metrics['Loss_val'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('loss')
plt.title('Loss During Training')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 学科二 输出 2 loss_during_training.png')
plt.show()
plt.figure(figsize=(14, 7))
plt.plot(metrics_beifen['Epoch'], metrics_beifen['MAE'], label='Train MAE')
plt.plot(metrics_beifen['Epoch'], metrics_beifen['MAE_val'], label='Validation MAE')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.title('MAE During Training')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 学科二 输出 3 MAE_during_training.png')
plt.show()
plt.figure(figsize=(14, 7))
plt.plot(metrics_beifen['Epoch'], metrics_beifen['RMSE'], label='Train RMSE')
plt.plot(metrics_beifen['Epoch'], metrics_beifen['RMSE_val'], label='Validation RMSE')
plt.xlabel('Epoch')
plt.ylabel('RMSE')
```

```
plt.title('RMSE During Training')
plt.legend()
plt.grid(True)
plt.savefig('demo11.2 NP 模型 2 学科二 输出 4 RMSE_during_training.png')
plt.show()
```

| 程序编号 | 5 | 程序说明 | 问题三：LSTM-NP 康复医学科三病房门诊收入预测模型 | 编程软件 | Python |
|---|---|---|---|---|---|

```
#%%
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np
import matplotlib.pyplot as plt
# 加载 LSTM 和 Neural Prophet 模型的预测结果
lstm_forecast = pd.read_excel('demo12 输出 学科三 LSTM 模型-康复医学科三 predicted_data 备份.xlsx')  # 这是 LSTM 的预测结果文件路径
Neural Prophet_forecast = pd.read_excel('demo12 NP 模型 2 学科三 输出预测数据 备份.xlsx')  # 这是 Neural Prophet 的预测结果文件路径
# 重命名列以确保匹配
lstm_forecast.rename(columns={'日期': 'ds', '门诊收入 OBS_T01_MZSR68': 'lstm_pred'}, inplace=True)
Neural Prophet_forecast.rename(columns={'yhat1': 'Neural Prophet_pred'}, inplace=True)
# 确保 'ds' 列是 datetime 类型
lstm_forecast['ds'] = pd.to_datetime(lstm_forecast['ds'])
Neural Prophet_forecast['ds'] = pd.to_datetime(Neural Prophet_forecast['ds'])
# 合并预测结果
merged_forecast = pd.merge(lstm_forecast[['ds', 'lstm_pred']],
                           Neural Prophet_forecast[['ds', 'Neural Prophet_pred']],
                           on='ds',
                           how='inner')
# 加载实际收入数据
actual_data = pd.read_excel('data.xlsx')  # 替换为你的实际文件路径
actual_data = actual_data[actual_data['当日病房收入对应科室'] == '康复医学科三病房']
actual_data['ds'] = pd.to_datetime(actual_data['日期'])
actual_data.rename(columns={'门诊收入 OBS_T01_MZSR68': 'actual'}, inplace=True)
# 合并实际数据和预测数据
final_merged_data = pd.merge(merged_forecast, actual_data[['ds', 'actual']], on='ds', how='left')
# 创建训练集、测试集和预测集
train_size = len(final_merged_data) - 30
test_size = 15
train_data = final_merged_data.iloc[:train_size]
test_data = final_merged_data.iloc[train_size:train_size + test_size]
predict_data = final_merged_data.iloc[train_size + test_size:]
# 填充预测集的实际值为 NaN
predict_data['actual'] = np.nan
# 构建训练集
X_train = train_data[['lstm_pred', 'Neural Prophet_pred']].values
y_train = train_data['actual'].values
# 构建测试集
X_test = test_data[['lstm_pred', 'Neural Prophet_pred']].values
y_test = test_data['actual'].values
# 训练线性回归模型
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# 预测
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
predict_predictions = model.predict(predict_data[['lstm_pred', 'Neural Prophet_pred']].values)
# 计算均方误差
train_mse = mean_squared_error(y_train, train_predictions)
test_mse = mean_squared_error(y_test, test_predictions)
print(f'Train MSE: {train_mse}')
print(f'Test MSE: {test_mse}')
# 将预测结果添加到预测集
predict_data['predicted'] = predict_predictions
# 绘制结果
plt.figure(figsize=(14, 7))
plt.plot(final_merged_data['ds'], final_merged_data['actual'], label='Actual')
plt.plot(train_data['ds'], train_predictions, label='Train Predictions')
plt.plot(test_data['ds'], test_predictions, label='Test Predictions')
plt.plot(predict_data['ds'], predict_predictions, label='Future Predictions')
plt.xlabel('Date')
plt.ylabel('Income')
plt.title('Actual vs Predicted Income using Ensemble Model')
plt.legend()
plt.grid(True)
plt.savefig('demo12 输出 学科三 2ensemble_model_predictions.png')
plt.show()
# 绘制损失函数
train_errors = (y_train - train_predictions) ** 2
test_errors = (y_test - test_predictions) ** 2
plt.figure(figsize=(14, 7))
plt.plot(train_data['ds'], train_errors, label='Train Loss')
plt.plot(test_data['ds'], test_errors, label='Test Loss')
plt.xlabel('Date')
plt.ylabel('Mean Squared Error')
plt.title('Loss over Time')
plt.legend()
plt.grid(True)
plt.savefig('demo12 输出 学科三 2loss_over_time.png')
plt.show()
# 输出预测结果
print("Future Predictions:")
print(predict_data[['ds', 'predicted']]
```